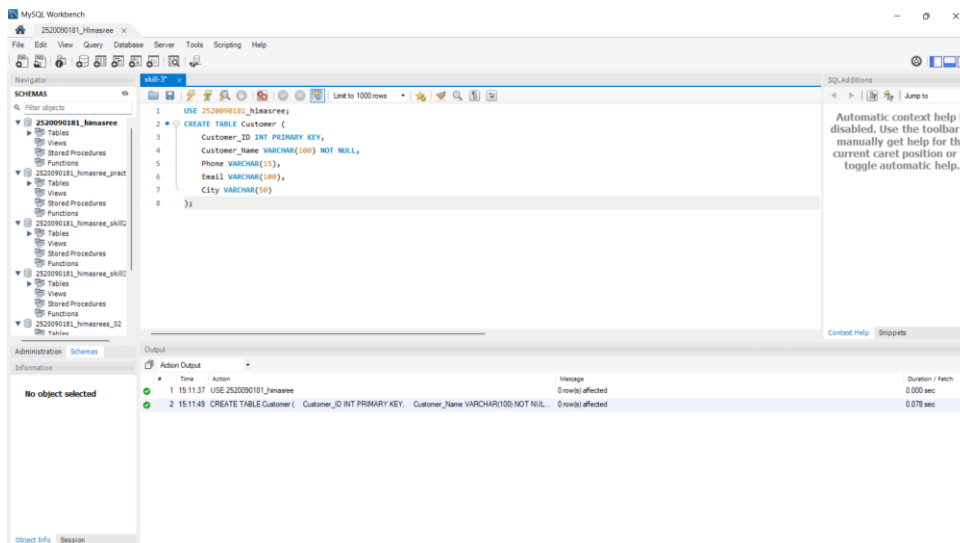
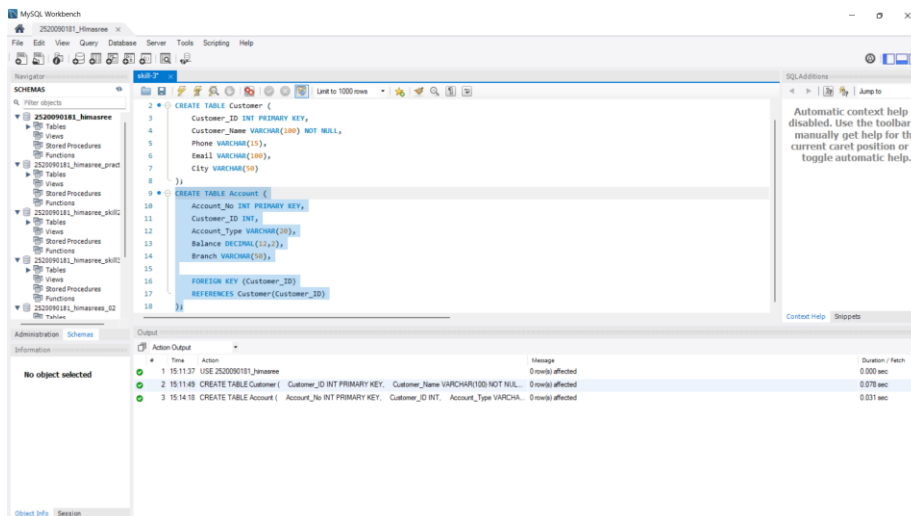


VIEWS

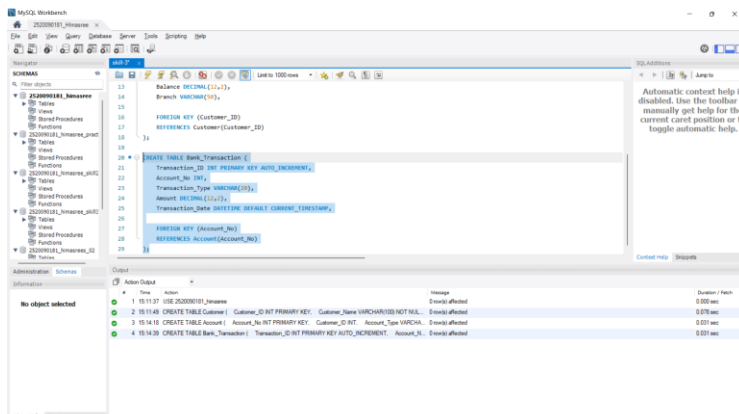
1. CREATE DATABASE / 2. CREATE CUSTOMER TABLE



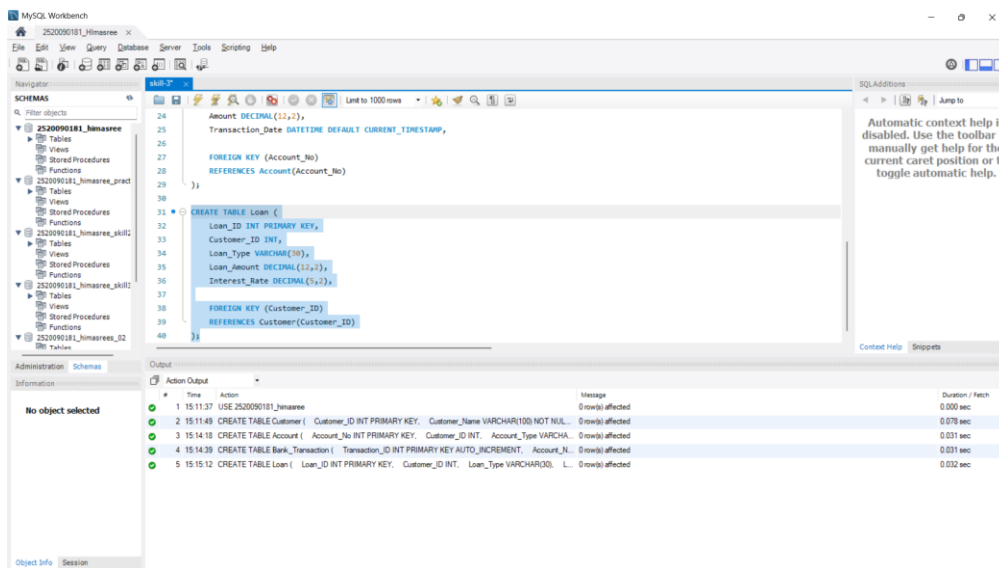
3. CREATE ACCOUNT TABLE



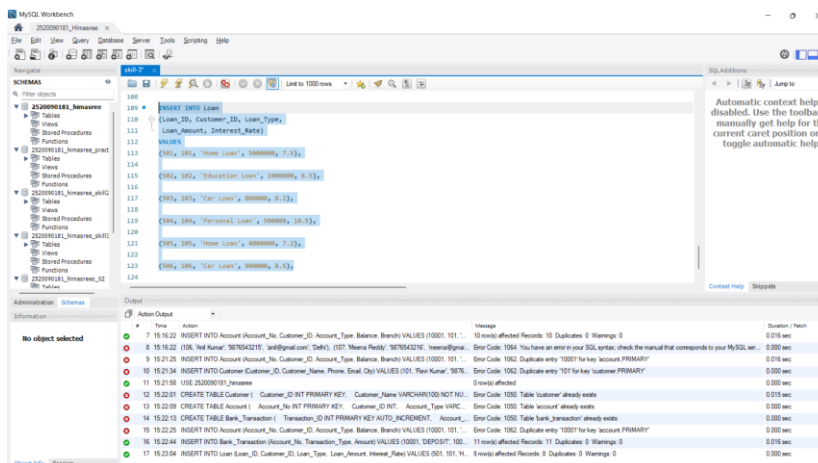
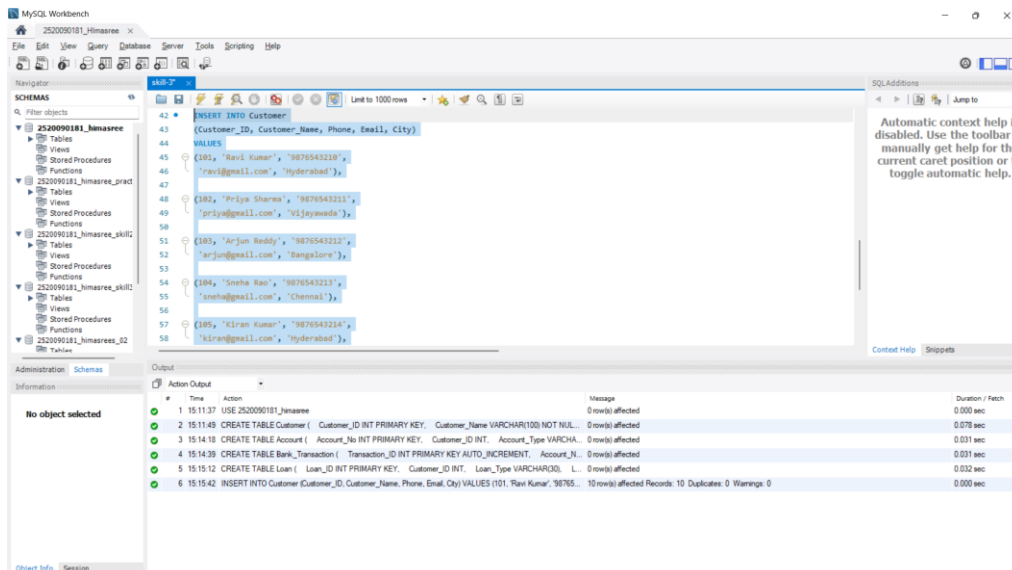
4. CREATE BANK TRANSACTION TABLE



5. CREATE LOAN TABLE



6-9. INSERT CUSTOMER DATA / ACCOUNT DATA / TRANSACTION DATA / LOAN DATA



[illegible]

The screenshot displays the MySQL Workbench environment. The top toolbar includes icons for file operations, database management, and query execution. The left sidebar shows a 'SCHEMAS' tree with a tree icon, listing databases like '252090181_hmsbase' and '252090181_hmsbase_sauc'. The main workspace is divided into three panes. The top pane shows a SQL query: `137`, `138 * DROP VIEW IF EXISTS Customer_View;`, `139 * CREATE VIEW Customer_View AS`, `140 SELECT *`, `141 FROM Customer;`, `142`, `143 * FROM Customer_View;`. The middle pane shows the 'Result Grid' with columns 'Customer_ID', 'Customer_Name', 'Phone', 'Email', and 'City', containing 10 rows of data. The bottom pane shows the 'Output' tab with 'Action Output' and 'Messages'. The 'Action Output' pane shows a list of actions with status icons (green for success, red for error) and timestamps. The 'Messages' pane shows error messages for failed actions, such as 'Error Code: 1054. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version 5.7.26: Table 'Customer_View' already exists'.

The screenshot displays the MySQL Workbench environment. The top toolbar includes icons for File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' tree with a filter for objects, listing databases like '2520090181_himansree' and '2520090181_himansree_skill2'. The main editor window shows a SQL script to create a view named 'Customer_Basic_View AS' with a SELECT statement from the 'Customer' table. The 'Result Grid' displays the data returned by the view, showing columns 'Customer_ID', 'Customer_Name', and 'City' for 10 rows. The bottom status bar indicates 'No object selected'.

13. CREATE A VIEW FOR ACCOUNT INFORMATION

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
153 SELECT
154     Account_No,
155     Account_Type,
156     Balance,
157     Branch
158 FROM Account;
159 SELECT * FROM Account_View;
```

The Result Grid displays the following data:

Account_No	Account_Type	Balance	Branch
10001	Savings	90000.00	Hyderabad
10002	Savings	75000.00	Vijayawada
10003	Current	120000.00	Bangalore
10004	Savings	40000.00	Chennai
10005	Current	90000.00	Hyderabad
10006	Savings	60000.00	Delhi
10007	Current	150000.00	Mumbai
10008	Savings	38000.00	Pune

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
22	15:26:34	CREATE VIEW Customer_View AS SELECT * FROM Customer	0 row(s) affected	0.016 sec
23	15:26:34	DISPLAY VIEW SELECT * FROM Customer_View	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' FROM Customer_View'	0.000 sec
24	15:30:48	CREATE VIEW Customer_View AS SELECT * FROM Customer	Error Code: 1050. Table 'Customer_View' already exists	0.000 sec
25	15:34:25	CREATE VIEW Customer_View AS SELECT * FROM Customer	Error Code: 1050. Table 'Customer_View' already exists	0.000 sec
26	15:34:39	DROP VIEW IF EXISTS Customer_View	0 row(s) affected	0.016 sec
27	15:34:42	CREATE VIEW Customer_View AS SELECT * FROM Customer	0 row(s) affected	0.016 sec
28	15:34:52	SELECT * FROM Customer_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	15:35:35	CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer	0 row(s) affected	0.047 sec
30	15:35:45	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	15:36:18	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ...	0 row(s) affected	0.031 sec
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

14. CREATE A VIEW FOR SAVINGS ACCOUNTS

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
161 CREATE VIEW Savings_Account_View AS
162 SELECT *
163 FROM Account
164 WHERE Account_Type = 'Savings';
165
166 SELECT * FROM Savings_Account_View;
```

The Result Grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	90000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10004	104	Savings	40000.00	Chennai
10006	106	Savings	60000.00	Delhi
10008	108	Savings	35000.00	Pune
10009	109	Savings	85000.00	Hyderabad

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
24	15:30:48	CREATE VIEW Customer_View AS SELECT * FROM Customer	Error Code: 1050. Table 'Customer_View' already exists	0.000 sec
25	15:34:25	CREATE VIEW Customer_View AS SELECT * FROM Customer	Error Code: 1050. Table 'Customer_View' already exists	0.000 sec
26	15:34:39	DROP VIEW IF EXISTS Customer_View	0 row(s) affected	0.016 sec
27	15:34:42	CREATE VIEW Customer_View AS SELECT * FROM Customer	0 row(s) affected	0.016 sec
28	15:34:52	SELECT * FROM Customer_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	15:35:35	CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer	0 row(s) affected	0.047 sec
30	15:35:45	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	15:36:18	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ...	0 row(s) affected	0.031 sec
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
33	15:36:56	CREATE VIEW Savings_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Savings'	0 row(s) affected	0.015 sec
34	15:37:08	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

15. CREATE A VIEW FOR CURRENT ACCOUNTS

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
167
168 CREATE VIEW Current_Account_View AS
169 SELECT *
170 FROM Account
171 WHERE Account_Type = 'Current';
172
173 SELECT * FROM Current_Account_View;
```

The Result Grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	90000.00	Hyderabad
10007	107	Current	150000.00	Mumbai
10010	110	Current	110000.00	Vijayawada

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
26	15:34:39	DROP VIEW IF EXISTS Customer_View	0 row(s) affected	0.016 sec
27	15:34:42	CREATE VIEW Customer_View AS SELECT * FROM Customer	0 row(s) affected	0.016 sec
28	15:34:52	SELECT * FROM Customer_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	15:35:35	CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer	0 row(s) affected	0.047 sec
30	15:35:45	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	15:36:18	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ...	0 row(s) affected	0.031 sec
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
33	15:36:56	CREATE VIEW Savings_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Savings'	0 row(s) affected	0.015 sec
34	15:37:08	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
35	15:37:52	CREATE VIEW Current_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Current'	0 row(s) affected	0.031 sec
36	15:38:01	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

16. HIGH BALANCE ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
177 Account_No,
178 Customer_ID,
179 Account_Type,
180 Balance
181 FROM Account
182 WHERE Balance > 100000;
183 SELECT * FROM High_Balance_View;
```

The Results tab displays the output of the query:

Account_No	Customer_ID	Account_Type	Balance
10001	101	Current	120000.00
10007	107	Current	130000.00
10010	110	Current	120000.00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
28	15:34:52	SELECT * FROM Customer_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
29	15:35:35	CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer	0 rows affected	0.047 sec
30	15:35:45	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
31	15:36:18	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM	0 rows affected	0.031 sec
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
33	15:36:56	CREATE VIEW Savings_Account_View AS SELECT FROM Account WHERE Account_Type = 'Savings'	0 rows affected	0.015 sec
34	15:37:08	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 rows returned	0.000 sec / 0.000 sec
35	15:37:52	CREATE VIEW Current_Account_View AS SELECT FROM Account WHERE Account_Type = 'Current'	0 rows affected	0.031 sec
36	15:38:01	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 rows returned	0.000 sec / 0.000 sec
37	15:38:31	CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan...	0 rows affected	0.016 sec
38	15:38:40	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 rows returned	0.000 sec / 0.000 sec

17. HYDERABAD ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
183 SELECT * FROM High_Balance_View;
184
185 CREATE VIEW Hyderabad_Account_View AS
186 SELECT *
187 FROM Account
188 WHERE Branch = 'Hyderabad';
189 SELECT * FROM Hyderabad_Account_View;
```

The Results tab displays the output of the query:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
30	15:35:45	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
31	15:36:18	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM	0 rows affected	0.031 sec
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
33	15:36:56	CREATE VIEW Savings_Account_View AS SELECT FROM Account WHERE Account_Type = 'Savings'	0 rows affected	0.015 sec
34	15:37:08	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 rows returned	0.000 sec / 0.000 sec
35	15:37:52	CREATE VIEW Current_Account_View AS SELECT FROM Account WHERE Account_Type = 'Current'	0 rows affected	0.031 sec
36	15:38:01	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 rows returned	0.000 sec / 0.000 sec
37	15:38:31	CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan...	0 rows affected	0.016 sec
38	15:38:40	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 rows returned	0.000 sec / 0.000 sec
39	15:39:07	CREATE VIEW Hyderabad_Account_View AS SELECT FROM Account WHERE Branch = 'Hyderabad'	0 rows affected	0.016 sec
40	15:39:16	SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000	3 rows returned	0.000 sec / 0.000 sec

18. LOW BALANCE ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following SQL script:

```
192 SELECT
193 Account_No,
194 Customer_ID,
195 Balance
196 FROM Account
197 WHERE Balance < 50000;
198 SELECT * FROM Low_Balance_View;
```

The Results tab displays the output of the query:

Account_No	Customer_ID	Balance
10004	104	40000.00
10008	108	30000.00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
32	15:36:28	SELECT * FROM Account_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
33	15:36:56	CREATE VIEW Savings_Account_View AS SELECT FROM Account WHERE Account_Type = 'Savings'	0 rows affected	0.015 sec
34	15:37:08	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 rows returned	0.000 sec / 0.000 sec
35	15:37:52	CREATE VIEW Current_Account_View AS SELECT FROM Account WHERE Account_Type = 'Current'	0 rows affected	0.031 sec
36	15:38:01	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 rows returned	0.000 sec / 0.000 sec
37	15:38:31	CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan...	0 rows affected	0.016 sec
38	15:38:40	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 rows returned	0.000 sec / 0.000 sec
39	15:39:07	CREATE VIEW Hyderabad_Account_View AS SELECT FROM Account WHERE Branch = 'Hyderabad'	0 rows affected	0.016 sec
40	15:39:16	SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000	3 rows returned	0.000 sec / 0.000 sec
41	15:39:43	CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account...	0 rows affected	0.016 sec
42	15:39:54	SELECT * FROM Low_Balance_View LIMIT 0, 1000	2 rows returned	0.000 sec / 0.000 sec

19. CUSTOMER + ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface with a query editor containing the following SQL code:

```
206 A.Account_Type,  
207 A.Balance,  
208 A.Branch  
209 FROM Customer C  
210 JOIN Account A  
211 ON C.Customer_ID = A.Customer_ID  
212 SELECT * FROM Customer_Account_View
```

The Results grid displays the following data:

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
102	Prave Sharma	Wayanada	10002	Savings	75000.00	Wayanada
103	Aryun Reddy	Bangalore	10003	Current	120000.00	Bangalore
104	Shritha Rao	Chennai	10004	Savings	45000.00	Chennai
105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
107	Meena Reddy	Mumbai	10007	Current	130000.00	Mumbai
108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune

The Output pane shows the execution log with the following messages:

```
34 15:37:08 SELECT * FROM Savings_Account_View LIMIT 0, 1000 6 rows returned 0.000 sec / 0.000 sec  
35 15:37:52 CREATE VIEW Current_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Current' 0 rows affected 0.011 sec  
36 15:38:01 SELECT * FROM Current_Account_View LIMIT 0, 1000 4 rows returned 0.000 sec / 0.000 sec  
37 15:38:31 CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan... 0 rows affected 0.016 sec  
38 15:38:40 SELECT * FROM High_Balance_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
39 15:39:07 CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad' 0 rows affected 0.016 sec  
40 15:39:16 SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
41 15:39:43 CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account... 0 rows affected 0.016 sec  
42 15:39:54 SELECT * FROM Low_Balance_View LIMIT 0, 1000 2 rows returned 0.000 sec / 0.000 sec  
43 15:40:25 CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A... 0 rows affected 0.016 sec  
44 15:40:35 SELECT * FROM Customer_Account_View LIMIT 0, 1000 10 rows returned 0.000 sec / 0.000 sec
```

20. HYDERABAD CUSTOMER ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface with a query editor containing the following SQL code:

```
219 A.Account_Type,  
220 A.Balance,  
221 FROM Customer C  
222 JOIN Account A  
223 ON C.Customer_ID = A.Customer_ID  
224 WHERE A.Branch = 'Hyderabad'  
225 SELECT * FROM Hyderabad_Customer_Accounts
```

The Results grid displays the following data:

Customer_Name	City	Account_No	Account_Type	Balance
Ravi Kumar	Hyderabad	10001	Savings	50000.00
Kiran Kumar	Hyderabad	10005	Current	90000.00
Lakshmi Devi	Hyderabad	10009	Savings	85000.00

The Output pane shows the execution log with the following messages:

```
36 15:38:01 SELECT * FROM Current_Account_View LIMIT 0, 1000 4 rows returned 0.000 sec / 0.000 sec  
37 15:38:31 CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan... 0 rows affected 0.016 sec  
38 15:38:40 SELECT * FROM High_Balance_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
39 15:39:07 CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad' 0 rows affected 0.016 sec  
40 15:39:16 SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
41 15:39:43 CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account... 0 rows affected 0.016 sec  
42 15:39:54 SELECT * FROM Low_Balance_View LIMIT 0, 1000 2 rows returned 0.000 sec / 0.000 sec  
43 15:40:25 CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A... 0 rows affected 0.016 sec  
44 15:40:35 SELECT * FROM Customer_Account_View LIMIT 0, 1000 10 rows returned 0.000 sec / 0.000 sec  
45 15:41:07 CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N... 0 rows affected 0.016 sec  
46 15:41:18 SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec
```

21. CUSTOMER LOAN VIEW

The screenshot shows the MySQL Workbench interface with a query editor containing the following SQL code:

```
233 L.Loan_Type,  
234 L.Loan_Amount,  
235 L.Interest_Rate  
236 FROM Customer C  
237 JOIN Loan L  
238 ON C.Customer_ID = L.Customer_ID  
239 SELECT * FROM Customer_Loan_View
```

The Results grid displays the following data:

Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
101	Ravi Kumar	Hyderabad	101	Home Loan	500000.00	7.50
102	Prave Sharma	Wayanada	102	Education Loan	100000.00	6.50
103	Aryun Reddy	Bangalore	103	Car Loan	80000.00	8.20
104	Shritha Rao	Chennai	104	Personal Loan	30000.00	10.50
105	Kiran Kumar	Hyderabad	105	Home Loan	400000.00	7.20
106	Anil Kumar	Delhi	106	Car Loan	90000.00	8.50
107	Meena Reddy	Mumbai	107	Business Loan	200000.00	9.80
109	Lakshmi Devi	Hyderabad	108	Personal Loan	40000.00	10.00

The Output pane shows the execution log with the following messages:

```
38 15:38:40 SELECT * FROM High_Balance_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
39 15:39:07 CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad' 0 rows affected 0.016 sec  
40 15:39:16 SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
41 15:39:43 CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account... 0 rows affected 0.016 sec  
42 15:39:54 SELECT * FROM Low_Balance_View LIMIT 0, 1000 2 rows returned 0.000 sec / 0.000 sec  
43 15:40:25 CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A... 0 rows affected 0.016 sec  
44 15:40:35 SELECT * FROM Customer_Account_View LIMIT 0, 1000 10 rows returned 0.000 sec / 0.000 sec  
45 15:41:07 CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N... 0 rows affected 0.016 sec  
46 15:41:18 SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0, 1000 3 rows returned 0.000 sec / 0.000 sec  
47 15:41:55 CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo... 0 rows affected 0.015 sec  
48 15:42:04 SELECT * FROM Customer_Loan_View LIMIT 0, 1000 8 rows returned 0.000 sec / 0.000 sec
```

22. COMPLETE CUSTOMER BANKING VIEW

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
253 L.Loan_Amount
254 FROM Customer C
255 LEFT JOIN Account A
256 ON C.Customer_ID = A.Customer_ID
257 LEFT JOIN Loan L
258 ON C.Customer_ID = L.Customer_ID
259 SELECT * FROM Customer_Banking_View
```

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch	Loan_Type	Loan_Amount
991	Nani Kumar	Hyderabad	20001	Savings	50000.00	Hyderabad	Home Loan	500000.00
102	Prave Sharma	Vijayawada	20002	Savings	70000.00	Vijayawada	Education Loan	100000.00
103	Arun Reddy	Bangalore	20003	Current	120000.00	Bangalore	Car Loan	80000.00
104	Rohini Rao	Chennai	20004	Savings	40000.00	Chennai	Personal Loan	60000.00
105	Kiran Kumar	Hyderabad	20005	Current	90000.00	Hyderabad	Home Loan	400000.00
106	Aad Kumar	Delhi	20006	Savings	60000.00	Delhi	Car Loan	90000.00
107	Renu Reddy	Mumbai	20007	Current	100000.00	Mumbai	Business Loan	300000.00
108	Rahul Sharma	Pune	20008	Savings	30000.00	Pune		

Output

#	Time	Action	Message	Duration / Fetch
40	15:39:16	SELECT * FROM Hyderabad_Account_View LIMIT 5: 1000	3 row(s) returned	0.000 sec / 0.000 sec
41	15:39:43	CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account	0 row(s) affected	0.016 sec
42	15:39:54	SELECT * FROM Low_Balance_View LIMIT 5: 1000	2 row(s) returned	0.000 sec / 0.000 sec
43	15:40:25	CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.	0 row(s) affected	0.016 sec
44	15:40:35	SELECT * FROM Customer_Account_View LIMIT 0: 1000	10 row(s) returned	0.000 sec / 0.000 sec
45	15:41:07	CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N.	0 row(s) affected	0.016 sec
46	15:41:18	SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0: 1000	3 row(s) returned	0.000 sec / 0.000 sec
47	15:41:55	CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo.	0 row(s) affected	0.015 sec
48	15:42:04	SELECT * FROM Customer_Loan_View LIMIT 0: 1000	8 row(s) returned	0.000 sec / 0.000 sec
49	15:42:41	CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.	0 row(s) affected	0.015 sec
50	15:42:51	SELECT * FROM Customer_Banking_View LIMIT 0: 1000	10 row(s) returned	0.000 sec / 0.000 sec
51	15:43:21	CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account	0 row(s) affected	0.016 sec
52	15:43:30	SELECT * FROM Total_Bank_Balance LIMIT 0: 1000	1 row(s) returned	0.016 sec / 0.000 sec

23. TOTAL BANK BALANCE

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
258
259
260 CREATE VIEW Total_Bank_Balance AS
261 SELECT
262 SUM(Balance) AS Total_Balance
263 FROM Account
264 SELECT * FROM Total_Bank_Balance
```

Total_Balance
823000.00

Output

#	Time	Action	Message	Duration / Fetch
42	15:39:44	SELECT * FROM Low_Balance_View LIMIT 0: 1000	2 row(s) returned	0.000 sec / 0.000 sec
43	15:40:25	CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.	0 row(s) affected	0.016 sec
44	15:40:35	SELECT * FROM Customer_Account_View LIMIT 0: 1000	10 row(s) returned	0.000 sec / 0.000 sec
45	15:41:07	CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N.	0 row(s) affected	0.016 sec
46	15:41:18	SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0: 1000	3 row(s) returned	0.000 sec / 0.000 sec
47	15:41:55	CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo.	0 row(s) affected	0.015 sec
48	15:42:04	SELECT * FROM Customer_Loan_View LIMIT 0: 1000	8 row(s) returned	0.000 sec / 0.000 sec
49	15:42:41	CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.	0 row(s) affected	0.015 sec
50	15:42:51	SELECT * FROM Customer_Banking_View LIMIT 0: 1000	10 row(s) returned	0.000 sec / 0.000 sec
51	15:43:21	CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account	0 row(s) affected	0.016 sec
52	15:43:30	SELECT * FROM Total_Bank_Balance LIMIT 0: 1000	1 row(s) returned	0.016 sec / 0.000 sec

24. AVERAGE ACCOUNT BALANCE

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
265
266 CREATE VIEW Average_Account_Balance AS
267 SELECT
268 AVG(Balance) AS Average_Balance
269 FROM Account
270
271 SELECT * FROM Average_Account_Balance
```

Average_Balance
82300.000000

Output

#	Time	Action	Message	Duration / Fetch
45	15:41:07	CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N.	0 row(s) affected	0.016 sec
46	15:41:18	SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0: 1000	3 row(s) returned	0.000 sec / 0.000 sec
47	15:41:55	CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo.	0 row(s) affected	0.015 sec
48	15:42:04	SELECT * FROM Customer_Loan_View LIMIT 0: 1000	8 row(s) returned	0.000 sec / 0.000 sec
49	15:42:41	CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.	0 row(s) affected	0.015 sec
50	15:42:51	SELECT * FROM Customer_Banking_View LIMIT 0: 1000	10 row(s) returned	0.000 sec / 0.000 sec
51	15:43:21	CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account	0 row(s) affected	0.016 sec
52	15:43:30	SELECT * FROM Total_Bank_Balance LIMIT 0: 1000	1 row(s) returned	0.016 sec / 0.000 sec
53	15:44:09	CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Acco.	0 row(s) affected	0.031 sec
54	15:45:26	CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Acco.	0 row(s) affected	0.000 sec
55	15:45:29	SELECT * FROM Average_Account_Balance LIMIT 0: 1000	1 row(s) returned	0.015 sec / 0.000 sec

25. MAXIMUM ACCOUNT BALANCE

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
271 SELECT * FROM Average_Account_Balance;
272
273 CREATE VIEW Maximum_Balance_View AS
274 SELECT
275 MAX(Balance) AS Maximum_Balance
276 FROM Account;
277 SELECT * FROM Maximum_Balance_View;
```

The Results grid shows the output of the last query:

Maximum_Balance
150000.00

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
47	15:41:55	CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Loan_Amount	0 rows affected	0.015 sec
48	15:42:04	SELECT * FROM Customer_Loan_View LIMIT 0, 1000	0 rows returned	0.000 sec / 0.000 sec
49	15:42:41	CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.Account	0 rows affected	0.015 sec
50	15:42:51	SELECT * FROM Customer_Banking_View LIMIT 0, 1000	10 rows returned	0.000 sec / 0.000 sec
51	15:43:21	CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account	0 rows affected	0.016 sec
52	15:43:30	SELECT * FROM Total_Bank_Balance LIMIT 0, 1000	1 row returned	0.016 sec / 0.000 sec
53	15:44:09	CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Account	0 rows affected	0.031 sec
54	15:45:26	CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Account	Error Code: 1050. Table 'Average_Account_Balance' already exists	0.000 sec
55	15:45:29	SELECT * FROM Average_Account_Balance LIMIT 0, 1000	1 row returned	0.015 sec / 0.000 sec
56	15:45:57	CREATE VIEW Maximum_Balance_View AS SELECT MAX(Balance) AS Maximum_Balance FROM Account	0 rows affected	0.015 sec
57	15:46:06	SELECT * FROM Maximum_Balance_View LIMIT 0, 1000	1 row returned	0.016 sec / 0.000 sec

26. MINIMUM ACCOUNT BALANCE

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
277 SELECT * FROM Maximum_Balance_View;
278
279 CREATE VIEW Minimum_Balance_View AS
280 SELECT
281 MIN(Balance) AS Minimum_Balance
282 FROM Account;
283 SELECT * FROM Minimum_Balance_View;
```

The Results grid shows the output of the last query:

Minimum_Balance
35000.00

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	17:46:31	CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account	Error Code: 1046. No database selected. Select the default DB to be used by double-clicking its name in the SCH...	0.000 sec
2	17:46:36	USE 2520090181_himansree	0 rows affected	0.000 sec
3	17:46:39	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL;	Error Code: 1050. Table 'customer' already exists	0.031 sec
4	17:46:39	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'S,	Error Code: 1062. Duplicate entry '10001' for key 'account PRIMARY'	0.047 sec
5	17:46:54	CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account	0 rows affected	0.031 sec
6	17:47:08	SELECT * FROM Minimum_Balance_View LIMIT 0, 1000	1 row returned	0.000 sec / 0.000 sec

27. TOTAL NUMBER OF ACCOUNTS

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
284
285
286 CREATE VIEW Account_Count_View AS
287 SELECT
288 COUNT(*) AS Total_Accounts
289 FROM Account;
290 SELECT * FROM Account_Count_View;
```

The Results grid shows the output of the last query:

Total_Accounts
10

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	17:46:31	CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account	Error Code: 1046. No database selected. Select the default DB to be used by double-clicking its name in the SCH...	0.000 sec
2	17:46:36	USE 2520090181_himansree	0 rows affected	0.000 sec
3	17:46:39	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL;	Error Code: 1050. Table 'customer' already exists	0.031 sec
4	17:46:39	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'S,	Error Code: 1062. Duplicate entry '10001' for key 'account PRIMARY'	0.047 sec
5	17:46:54	CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account	0 rows affected	0.031 sec
6	17:47:08	SELECT * FROM Minimum_Balance_View LIMIT 0, 1000	1 row returned	0.000 sec / 0.000 sec
7	17:47:39	CREATE VIEW Account_Count_View AS SELECT COUNT(*) AS Total_Accounts FROM Account	0 rows affected	0.016 sec
8	17:47:44	SELECT * FROM Account_Count_View LIMIT 0, 1000	1 row returned	0.000 sec / 0.000 sec

28. BRANCH-WISE ACCOUNT COUNT

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
292 CREATE VIEW Branch_Account_Count AS
293 SELECT
294     Branch,
295     COUNT(*) AS Number_of_Accounts
296 FROM Account
297 GROUP BY Branch;
298 SELECT * FROM Branch_Account_Count;
```

The Result Grid shows the following data:

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	2
Bangalore	1
Chennai	1
Delhi	1
Mumbai	1
Pune	1

The Action Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
1	21:59:21	CREATE VIEW Branch_Account_Count AS SELEC...	Error Code: 1046: No database selected Select the ...	0.000 sec
2	21:59:31	USE 2520090181_himasee	0 row(s) affected	0.000 sec
3	21:59:43	INSERT INTO Customer (Customer_ID, Customer_...	Error Code: 1062: Duplicate entry '101' for key 'cust...	0.016 sec
4	21:59:52	CREATE VIEW Branch_Account_Count AS SELEC...	0 row(s) affected	0.015 sec
5	22:00:02	SELECT * FROM Branch_Account_Count LIMIT 0 ...	7 row(s) returned	0.000 sec / 0.000 sec

29. BRANCH-WISE TOTAL BALANCE

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
300 CREATE VIEW Branch_Total_Balance AS
301 SELECT
302     Branch,
303     SUM(Balance) AS Total_Balance
304 FROM Account
305 GROUP BY Branch;
306 SELECT * FROM Branch_Total_Balance;
```

The Result Grid shows the following data:

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Chennai	40000.00
Delhi	65000.00
Mumbai	150000.00
Pune	35000.00

The Action Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
1	21:59:21	CREATE VIEW Branch_Account_Count AS SELEC...	Error Code: 1046: No database selected Select the ...	0.000 sec
2	21:59:31	USE 2520090181_himasee	0 row(s) affected	0.000 sec
3	21:59:43	INSERT INTO Customer (Customer_ID, Customer_...	Error Code: 1062: Duplicate entry '101' for key 'cust...	0.016 sec
4	21:59:52	CREATE VIEW Branch_Account_Count AS SELEC...	0 row(s) affected	0.015 sec
5	22:00:02	SELECT * FROM Branch_Account_Count LIMIT 0 ...	7 row(s) returned	0.000 sec / 0.000 sec
6	22:00:41	CREATE VIEW Branch_Total_Balance AS SELEC...	0 row(s) affected	0.000 sec
7	22:00:50	SELECT * FROM Branch_Total_Balance LIMIT 0 ...	7 row(s) returned	0.000 sec / 0.000 sec

30. ACCOUNT TYPE COUNT

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
309 CREATE VIEW Account_Type_Count AS
310 SELECT
311     Account_Type,
312     COUNT(*) AS Number_of_Accounts
313 FROM Account
314 GROUP BY Account_Type;
315 SELECT * FROM Account_Type_Count;
```

The Result Grid shows the following data:

Account_Type	Number_of_Accounts
Savings	6
Current	4

The Action Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
1	21:59:21	CREATE VIEW Branch_Account_Count AS SELEC...	Error Code: 1046: No database selected Select the ...	0.000 sec
2	21:59:31	USE 2520090181_himasee	0 row(s) affected	0.000 sec
3	21:59:43	INSERT INTO Customer (Customer_ID, Customer_...	Error Code: 1062: Duplicate entry '101' for key 'cust...	0.016 sec
4	21:59:52	CREATE VIEW Branch_Account_Count AS SELEC...	0 row(s) affected	0.015 sec
5	22:00:02	SELECT * FROM Branch_Account_Count LIMIT 0 ...	7 row(s) returned	0.000 sec / 0.000 sec
6	22:00:41	CREATE VIEW Branch_Total_Balance AS SELEC...	0 row(s) affected	0.000 sec
7	22:00:50	SELECT * FROM Branch_Total_Balance LIMIT 0 ...	7 row(s) returned	0.000 sec / 0.000 sec
8	22:01:55	CREATE VIEW Account_Type_Count AS SELEC...	0 row(s) affected	0.016 sec
9	22:02:03	SELECT * FROM Account_Type_Count LIMIT 0.1 ...	2 row(s) returned	0.000 sec / 0.000 sec

31. ACCOUNT TYPE TOTAL BALANCE

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
317 CREATE VIEW Account_Type_Balance AS
318 SELECT
319     Account_Type,
320     SUM(Balance) AS Total_Balance
321 FROM Account_Type
322 GROUP BY Account_Type;
323 SELECT * FROM Account_Type_Balance;
```

The Results grid shows the output of the query:

Account_Type	Total_Balance
Savings	20000.00
Current	40000.00

The Action Output pane shows the execution of the view creation and the subsequent query.

32. BRANCHES WITH MORE THAN ONE ACCOUNT

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
326 SELECT
327     Branch,
328     COUNT(*) AS Number_of_Accounts
329 FROM Account_Type
330 GROUP BY Branch
331 HAVING COUNT(*) > 1;
332 SELECT * FROM Multiple_Account_Branches;
```

The Results grid shows the output of the query:

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	2

The Action Output pane shows the execution of the view creation and the subsequent query.

33. BRANCHES WITH TOTAL BALANCE ABOVE 100000

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
335 SELECT
336     Branch,
337     SUM(Balance) AS Total_Balance
338 FROM Account_Type
339 GROUP BY Branch
340 HAVING SUM(Balance) > 100000;
341 SELECT * FROM Rich_Branches;
```

The Results grid shows the output of the query:

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	85000.00
Bangalore	120000.00
Mumbai	150000.00

The Action Output pane shows the execution of the view creation and the subsequent query.

34. DEPOSIT TRANSACTION VIEW

The screenshot shows the MySQL Workbench interface with the 'Schemas' pane on the left displaying the '2520090181_himansree' database. The 'SQL Editor' pane contains the following SQL code:

```
341 * SELECT * FROM Rich_Branches;
342
343 CREATE VIEW Deposit_Transaction_View AS
344 SELECT *
345 FROM Bank_Transaction
346 WHERE Transaction_Type = 'DEPOSIT';
347 * SELECT * FROM Deposit_Transaction_View;
```

The 'Result Grid' pane shows the output of the final query, displaying a table with 10 rows of deposit transactions. The 'Output' pane shows the execution log, including the creation of the view.

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-12 15:22+
3	10002	DEPOSIT	15000.00	2026-08-12 15:22+
5	10004	DEPOSIT	5000.00	2026-08-12 15:22+
7	10006	DEPOSIT	12000.00	2026-08-12 15:22+
8	10007	DEPOSIT	25000.00	2026-08-12 15:22+
10	10009	DEPOSIT	20000.00	2026-08-12 15:22+

35. WITHDRAWAL TRANSACTION VIEW

The screenshot shows the MySQL Workbench interface with the 'Schemas' pane on the left displaying the '2520090181_himansree' database. The 'SQL Editor' pane contains the following SQL code:

```
347 * SELECT * FROM Deposit_Transaction_View
348
349 CREATE VIEW Withdrawal_Transaction_View AS
350 SELECT *
351 FROM Bank_Transaction
352 WHERE Transaction_Type = 'WITHDRAW';
353 * SELECT * FROM Withdrawal_Transaction_View;
```

The 'Result Grid' pane shows the output of the final query, displaying a table with 11 rows of withdrawal transactions. The 'Output' pane shows the execution log, including the creation of the view.

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
2	10001	WITHDRAW	5000.00	2026-08-12 15:22+
4	10003	WITHDRAW	20000.00	2026-08-12 15:22+
6	10005	WITHDRAW	8000.00	2026-08-12 15:22+
9	10008	WITHDRAW	5000.00	2026-08-12 15:22+
11	10010	WITHDRAW	15000.00	2026-08-12 15:22+

36. TRANSACTION DETAILS WITH CUSTOMER

The screenshot shows the MySQL Workbench interface with the 'Schemas' pane on the left displaying the '2520090181_himansree' database. The 'SQL Editor' pane contains the following SQL code:

```
363 * T.Transaction_Date
364 FROM Customer C
365 JOIN Account A
366 ON C.Customer_ID = A.Customer_ID
367 JOIN Bank_Transaction T
368 ON A.Account_No = T.Account_No;
369 * SELECT * FROM Customer_Transaction_View;
```

The 'Result Grid' pane shows the output of the final query, displaying a table with 11 rows of transaction details for each customer. The 'Output' pane shows the execution log, including the creation of the view.

Customer_Name	Account_No	Account_Type	Transaction_ID	Transaction_Type
Ravi Kumar	10001	Savings	1	DEPOSIT
Ravi Kumar	10001	Savings	2	WITHDRAW
Prisha Sharma	10002	Savings	3	DEPOSIT
Akshu Baddi	10003	Current	4	WITHDRAW
Shreha Rao	10004	Savings	5	DEPOSIT
Kiran Kumar	10005	Current	6	WITHDRAW

37. HIGH VALUE TRANSACTION VIEW

The screenshot shows the MySQL Workbench interface with a SQL editor window titled 'Multi-7'. The SQL code defines a view named 'High_Value_Transaction_View' that selects transactions with an amount greater than 10000. The 'Result Grid' shows the first few rows of the view's data.

```
370
371 CREATE VIEW High_Value_Transaction_View AS
372 SELECT *
373 FROM Bank_Transaction
374 WHERE Amount > 10000;
375
376 SELECT * FROM High_Value_Transaction_View;
```

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
3	10002	DEPOSIT	15000.00	2026-08-12 15:22+
4	10003	WITHDRAWAL	20000.00	2026-08-12 15:22+
7	10006	DEPOSIT	12000.00	2026-08-12 15:22+
8	10007	DEPOSIT	20000.00	2026-08-12 15:22+
10	10009	DEPOSIT	20000.00	2026-08-12 15:22+
11	10010	WITHDRAWAL	15000.00	2026-08-12 15:22+

The 'Action Output' pane shows the execution of the SQL statements, including the creation of the view and the subsequent SELECT query.

38. ACCOUNTS ORDERED BY BALANCE

The screenshot shows the MySQL Workbench interface with a SQL editor window titled 'Multi-7'. The SQL code defines a view named 'Balance_Ranking_View' that selects accounts ordered by their balance in descending order. The 'Result Grid' shows the first few rows of the view's data.

```
381 Account_No,
382 Customer_ID,
383 Account_Type,
384 Balance,
385 FROM Account
386 ORDER BY Balance DESC;
387 SELECT * FROM Balance_Ranking_View;
```

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00
10003	103	Current	120000.00
10010	110	Current	110000.00
10009	109	Current	90000.00
10009	109	Savings	85000.00
10002	102	Savings	75000.00
10006	106	Savings	65000.00
10004	104	Fixed	60000.00

The 'Action Output' pane shows the execution of the SQL statements, including the creation of the view and the subsequent SELECT query.

39. CUSTOMERS ORDERED BY NAME

The screenshot shows the MySQL Workbench interface with a SQL editor window titled 'Multi-7'. The SQL code defines a view named 'Customer_Name_View' that selects customers ordered by their name. The 'Result Grid' shows the first few rows of the view's data.

```
391 Customer_ID,
392 Customer_Name,
393 City,
394 FROM Customer
395 ORDER BY Customer_Name;
396
397 SELECT * FROM Customer_Name_View;
```

Customer_ID	Customer_Name	City
106	Anil Kumar	Delhi
103	Arun Reddy	Bangalore
105	Vijay Kumar	Hyderabad
109	Lakshmi Devi	Hyderabad
107	Meena Reddy	Mumbai
102	Priya Sharma	Hyderabad
108	Rahul Sharma	Pune

The 'Action Output' pane shows the execution of the SQL statements, including the creation of the view and the subsequent SELECT query.

40. LOAN INTEREST CALCULATION VIEW

The screenshot shows the MySQL Workbench interface with the 'Loan_Interest_View' being created. The SQL editor contains the following code:

```
403 Loan_Type,
404 Loan_Amount,
405 Interest_Rate,
406 (Loan_Amount * Interest_Rate / 100)
407 AS Annual_Interest
408 FROM Loan;
409 SELECT * FROM Loan_Interest_View;
```

The 'Result Grid' shows the following data:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Annual_Interest
901	101	Home Loan	500000.00	7.50	37500
902	102	Education Loan	100000.00	6.50	6500
903	103	Car Loan	80000.00	8.20	6560
904	104	Personal Loan	50000.00	10.50	5250
905	105	Home Loan	400000.00	7.20	28800
906	106	Car Loan	90000.00	8.50	7650

The 'Output' pane shows the execution log for the 'Loan_Interest_View' creation.

#	Time	Action	Message	Duration / Fetch
19	22:06:34	SELECT * FROM Withdrawal_Transaction_View ...	5 row(s) returned	0.000 sec / 0.000 sec
20	22:07:19	CREATE VIEW Customer_Transaction_View AS ...	0 row(s) affected	0.016 sec
22	22:08:24	CREATE VIEW High_Value_Transaction_View AS ...	0 row(s) affected	0.016 sec / 0.000 sec
23	22:08:34	SELECT * FROM High_Value_Transaction_View ...	6 row(s) returned	0.016 sec / 0.000 sec
24	22:08:58	CREATE VIEW Balance_Ranking_View AS SELECT ...	0 row(s) affected	0.015 sec
25	22:09:06	SELECT * FROM Balance_Ranking_View LIMIT ...	10 row(s) returned	0.000 sec / 0.000 sec
26	22:09:48	CREATE VIEW Customer_Name_View AS SELECT ...	0 row(s) affected	0.016 sec
27	22:09:57	SELECT * FROM Customer_Name_View LIMIT 0 ...	10 row(s) returned	0.016 sec / 0.000 sec
28	22:10:48	CREATE VIEW Loan_Interest_View AS SELECT ...	0 row(s) affected	0.016 sec
29	22:10:55	SELECT * FROM Loan_Interest_View LIMIT 0.1 ...	8 row(s) returned	0.016 sec / 0.000 sec

41. LOAN TOTAL AMOUNT VIEW

The screenshot shows the MySQL Workbench interface with the 'Loan_Total_Amount_View' being created. The SQL editor contains the following code:

```
418 Interest_Rate,
419 Loan_Amount +
420 (Loan_Amount * Interest_Rate / 100)
421 AS Total_Amount
422 FROM Loan;
423 SELECT * FROM Loan_Total_Amount_View;
```

The 'Result Grid' shows the following data:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Total
901	101	Home Loan	500000.00	7.50	53750
902	102	Education Loan	100000.00	6.50	10650
903	103	Car Loan	80000.00	8.20	86560
904	104	Personal Loan	50000.00	10.50	55250
905	105	Home Loan	400000.00	7.20	42880
906	106	Car Loan	90000.00	8.50	97650

The 'Output' pane shows the execution log for the 'Loan_Total_Amount_View' creation.

#	Time	Action	Message	Duration / Fetch
21	22:07:30	SELECT * FROM Customer_Transaction_View LI ...	11 row(s) returned	0.016 sec / 0.000 sec
22	22:08:24	CREATE VIEW High_Value_Transaction_View AS ...	0 row(s) affected	0.016 sec
23	22:08:34	SELECT * FROM High_Value_Transaction_View ...	6 row(s) returned	0.016 sec / 0.000 sec
24	22:08:58	CREATE VIEW Balance_Ranking_View AS SEL...	0 row(s) affected	0.015 sec
25	22:09:06	SELECT * FROM Balance_Ranking_View LIMIT ...	10 row(s) returned	0.000 sec / 0.000 sec
26	22:09:48	CREATE VIEW Customer_Name_View AS SELE...	0 row(s) affected	0.016 sec
27	22:09:57	SELECT * FROM Customer_Name_View LIMIT 0 ...	10 row(s) returned	0.016 sec / 0.000 sec
28	22:10:48	CREATE VIEW Loan_Interest_View AS SELECT ...	0 row(s) affected	0.016 sec
29	22:10:55	SELECT * FROM Loan_Interest_View LIMIT 0.1 ...	8 row(s) returned	0.016 sec / 0.000 sec
30	22:11:36	CREATE VIEW Loan_Total_Amount_View AS SE...	0 row(s) affected	0.016 sec
31	22:11:44	SELECT * FROM Loan_Total_Amount_View LIM...	8 row(s) returned	0.000 sec / 0.000 sec

42. DISPLAY HIGH BALANCE ACCOUNTS FROM VIEW

The screenshot shows the MySQL Workbench interface with the 'High_Balance_View' being created. The SQL editor contains the following code:

```
419 Loan_Amount +
420 (Loan_Amount * Interest_Rate / 100)
421 AS Total_Amount
422 FROM Loan;
423 SELECT * FROM Loan_Total_Amount_View;
424 SELECT *
425 FROM High_Balance_View;
```

The 'Result Grid' shows the following data:

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00

The 'Output' pane shows the execution log for the 'High_Balance_View' creation.

#	Time	Action	Message	Duration / Fetch
22	22:08:24	CREATE VIEW High_Value_Transaction_View AS ...	0 row(s) affected	0.015 sec
23	22:08:34	SELECT * FROM High_Value_Transaction_View ...	6 row(s) returned	0.016 sec / 0.000 sec
24	22:08:58	CREATE VIEW Balance_Ranking_View AS SEL...	0 row(s) affected	0.015 sec
25	22:09:06	SELECT * FROM Balance_Ranking_View LIMIT ...	10 row(s) returned	0.000 sec / 0.000 sec
26	22:09:48	CREATE VIEW Customer_Name_View AS SELE...	0 row(s) affected	0.016 sec
27	22:09:57	SELECT * FROM Customer_Name_View LIMIT 0 ...	10 row(s) returned	0.016 sec / 0.000 sec
28	22:10:48	CREATE VIEW Loan_Interest_View AS SELECT ...	0 row(s) affected	0.016 sec
30	22:11:36	CREATE VIEW Loan_Total_Amount_View AS SE...	0 row(s) affected	0.016 sec
31	22:11:44	SELECT * FROM Loan_Total_Amount_View LIM...	8 row(s) returned	0.000 sec / 0.000 sec
32	22:12:22	SELECT * FROM High_Balance_View WHERE B...	1 row(s) returned	0.000 sec / 0.000 sec

43. DISPLAY SAVINGS ACCOUNTS ABOVE 60000

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
SELECT * FROM Loan_Total_Amount_View  
SELECT *  
FROM High_Balance_View  
WHERE Balance > 120000  
SELECT *
```

The Results grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10002	102	Savings	75000.00	Vijayawada
10006	106	Savings	65000.00	Delhi
10009	109	Savings	85000.00	Hyderabad

The Action Output pane shows the execution of the query, indicating that 3 rows were returned.

44. DISPLAY HYDERABAD ACCOUNTS ABOVE 50000

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
FROM High_Balance_View  
WHERE Balance > 120000  
SELECT *  
FROM Savings_Account_View  
WHERE Balance > 60000
```

The Results grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad

The Action Output pane shows the execution of the query, indicating that 2 rows were returned.

45. DISPLAY CUSTOMERS FROM HYDERABAD

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
WHERE Balance > 60000  
[Execute the selected portion of the script or everything, if there is no selection]  
SELECT *  
FROM Hyderabad_Account_View  
WHERE Balance > 50000  
SELECT *
```

The Results grid displays the following data:

Customer_ID	Customer_Name	City
101	Ravi Kumar	Hyderabad
105	Kiran Kumar	Hyderabad
109	Lakshmi Devi	Hyderabad

The Action Output pane shows the execution of the query, indicating that 3 rows were returned.

46. DISPLAY CUSTOMER ACCOUNTS ORDERED BY BALANCE

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
435 WHERE Balance > 50000;
436
437 SELECT *
438 FROM Customer_Basic_View
439 WHERE City = 'Hyderabad';
440
441 SELECT *
```

The Results grid displays the following data:

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance
107	Meena Reddy	Mumbai	10007	Current	15000
101	Arun Reddy	Bangalore	10001	Current	12000
110	Suresh Babu	Vijayawada	10010	Current	11000
105	Kiran Kumar	Hyderabad	10005	Current	9000
109	Lakshmi Devi	Hyderabad	10009	Savings	8500
102	Priya Sharma	Vijayawada	10002	Savings	7500
106	Santhosh Kumar	Hyderabad	10006	Current	6000

The Action Output pane shows the execution of the query, indicating that 10 rows were returned.

47. DISPLAY LOANS ABOVE 1000000

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
441 SELECT *
442 FROM Customer_Account_View
443 ORDER BY Balance DESC;
444
445 SELECT *
```

The Results grid displays the following data:

Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount
101	Kiran Kumar	Hyderabad	101	Home Loan	3000000
105	Kiran Kumar	Hyderabad	105	Home Loan	4000000
107	Meena Reddy	Mumbai	107	Business Loan	3000000

The Action Output pane shows the execution of the query, indicating that 3 rows were returned.

48. UPDATE ACCOUNT BALANCE THROUGH VIEW

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
447 FROM Customer_Loan_View
448 WHERE Loan_Amount > 1000000;
449
450 UPDATE Account_View
451 SET Balance = 10000;
452 WHERE Account_No = 10001;
453 SELECT *
```

The Results grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10009	109	Savings	8500.00	Hyderabad

The Action Output pane shows the execution of the query, indicating that 1 row was affected.

49. INSERT USING SIMPLE VIEW

MySQL Workbench interface showing a query window with the following SQL code:

```
INSERT INTO Simple_Account_View  
VALUES  
(10011, 101, 'Savings', 55000, 'Hyderabad');
```

The query is executed, and the output shows the result grid with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Delhi
10007	107	Current	150000.00	Mumbai

The output also shows the action output with the following message:

```
42 22:18:13 INSERT INTO Simple_Account_View VALUES (1... 1 row(s) affected 0.016 sec
```

50. DELETE USING SIMPLE VIEW

MySQL Workbench interface showing a query window with the following SQL code:

```
DELETE FROM Simple_Account_View  
WHERE Account_No = 10011;
```

The query is executed, and the output shows the result grid with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Delhi
10007	107	Current	150000.00	Mumbai

The output also shows the action output with the following message:

```
44 22:19:13 DELETE FROM Simple_Account_View WHERE ... 1 row(s) affected 0.000 sec
```

51. SHOW ALL VIEWS

MySQL Workbench interface showing a query window with the following SQL code:

```
SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
```

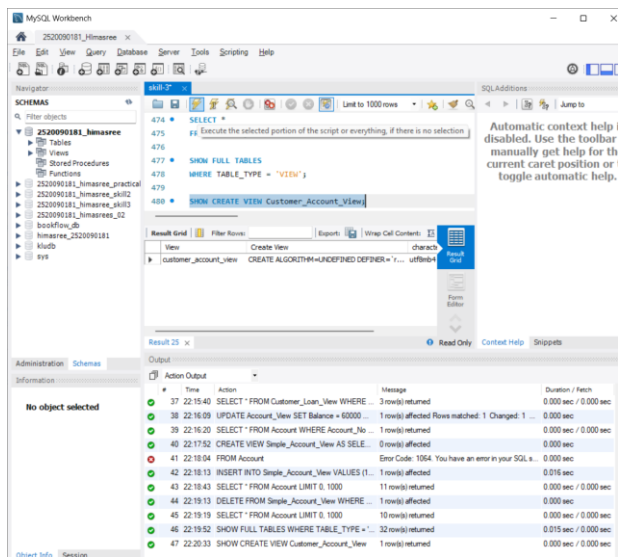
The query is executed, and the output shows the result grid with the following data:

Table_name	Table_type
account_count_view	VIEW
account_type_balance	VIEW
account_type_count	VIEW
account_view	VIEW
average_account_balance	VIEW
balance_saving_view	VIEW
branch_account_count	VIEW

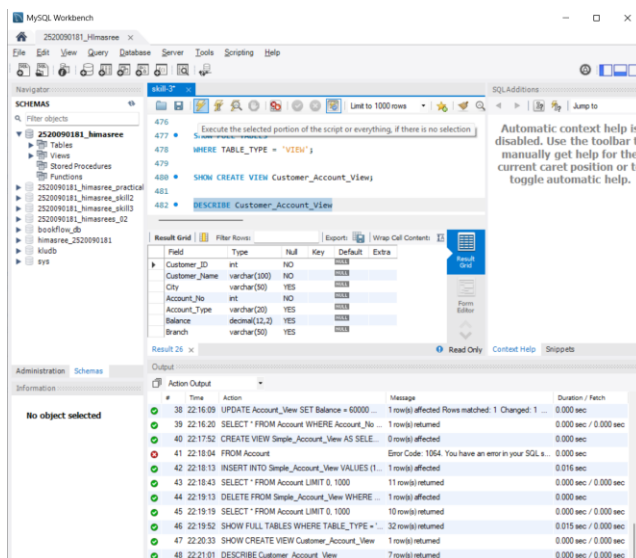
The output also shows the action output with the following message:

```
46 22:19:52 SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW' ... 32 row(s) returned 0.015 sec / 0.000 sec
```

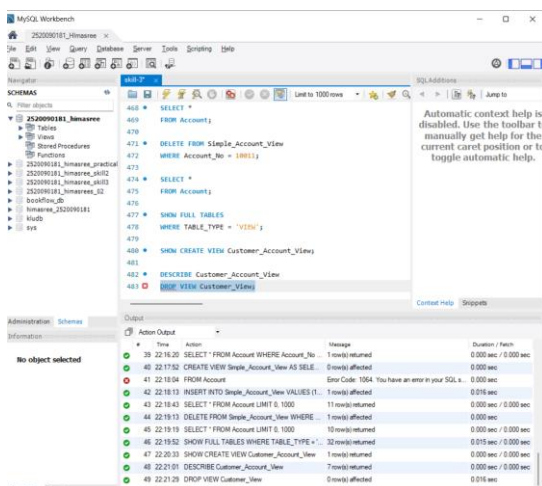
52. SHOW VIEW DEFINITION



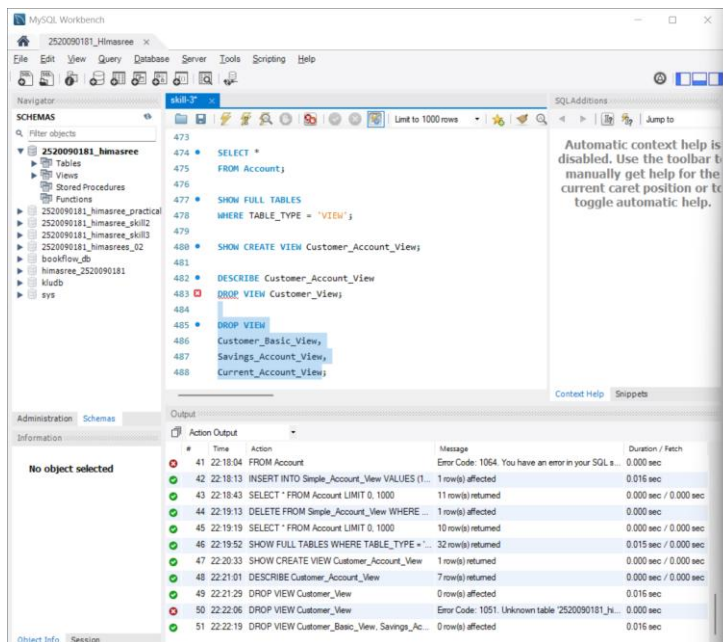
53. DESCRIBE VIEW



54. DROP ONE VIEW



55. DROP MULTIPLE VIEWS



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

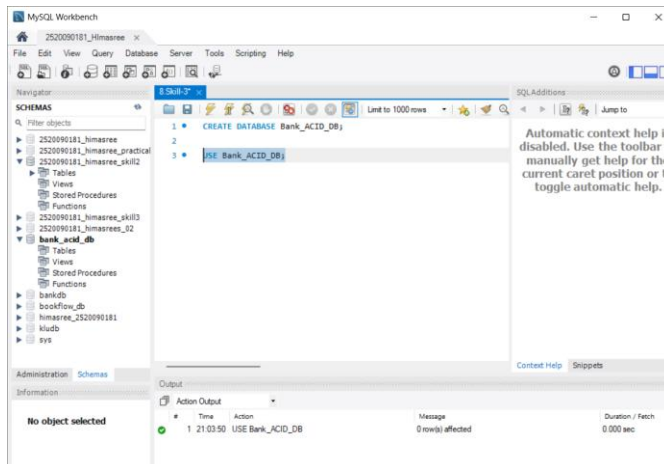
```
473 SELECT *
474 FROM Account;
475
476 SHOW FULL TABLES
477 WHERE TABLE_TYPE = 'VIEW';
478
479 SHOW CREATE VIEW Customer_Account_View;
480
481 DESCRIBE Customer_Account_View;
482
483 DROP VIEW Customer_View;
484
485 DROP VIEW
486 Customer_Basic_View,
487 Savings_Account_View,
488 Current_Account_View;
```

The output window shows the execution results:

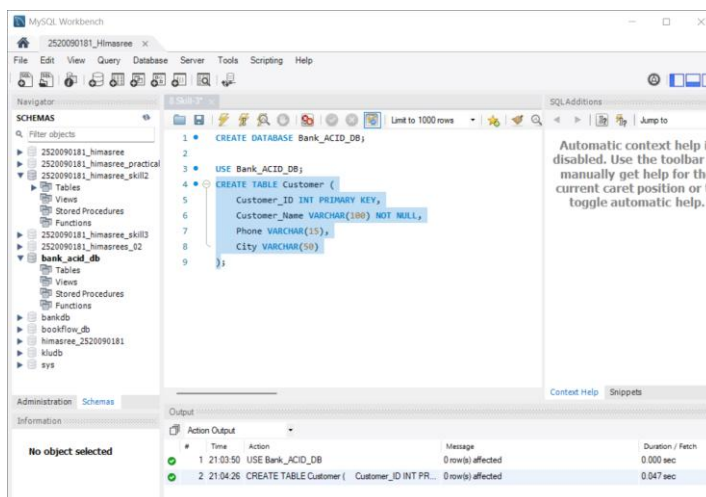
#	Time	Action	Message	Duration / Fetch
41	22:18:04	FROM Account	Error Code: 1054. You have an error in your SQL s...	0.000 sec
42	22:18:13	INSERT INTO Simple_Account_View VALUES (1...	1 row(s) affected	0.016 sec
43	22:18:43	SELECT * FROM Account LIMIT 0. 1000	11 row(s) returned	0.000 sec / 0.000 sec
44	22:19:13	DELETE FROM Simple_Account_View WHERE ...	1 row(s) affected	0.000 sec
45	22:19:19	SELECT * FROM Account LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec
46	22:19:52	SHOW FULL TABLES WHERE TABLE_TYPE = ...	32 row(s) returned	0.015 sec / 0.000 sec
47	22:20:33	SHOW CREATE VIEW Customer_Account_View	1 row(s) returned	0.000 sec / 0.000 sec
48	22:21:01	DESCRIBE Customer_Account_View	7 row(s) returned	0.000 sec / 0.000 sec
49	22:21:29	DROP VIEW Customer_View	0 row(s) affected	0.016 sec
50	22:22:06	DROP VIEW Customer_View	Error Code: 1051. Unknown table '2520090181_H...	0.000 sec
51	22:22:19	DROP VIEW Customer_Basic_View, Savings_Ac...	0 row(s) affected	0.016 sec

Skill-ACID Properties

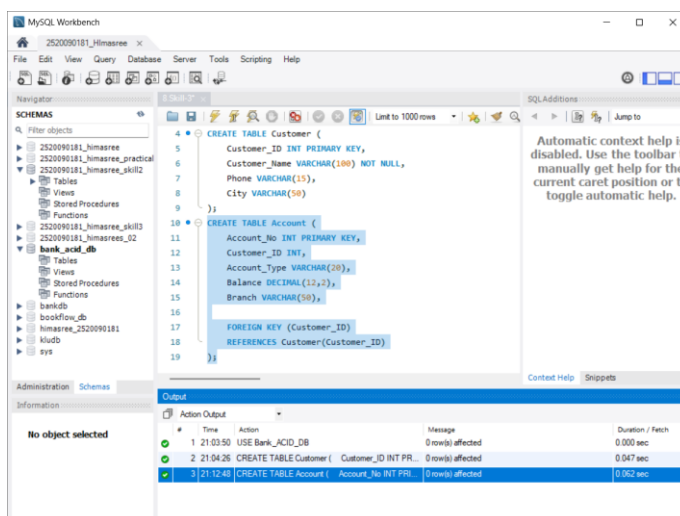
1. CREATE DATABASE



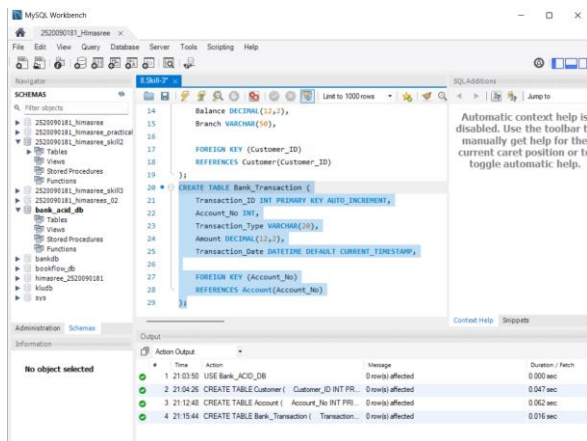
2. CREATE CUSTOMER TABLE



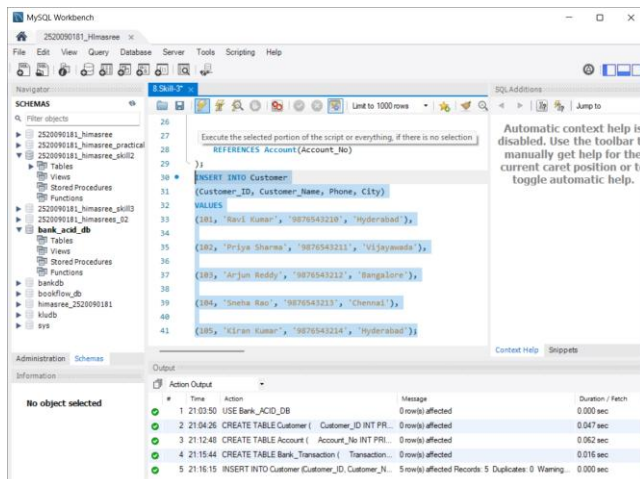
3. CREATE ACCOUNT TABLE



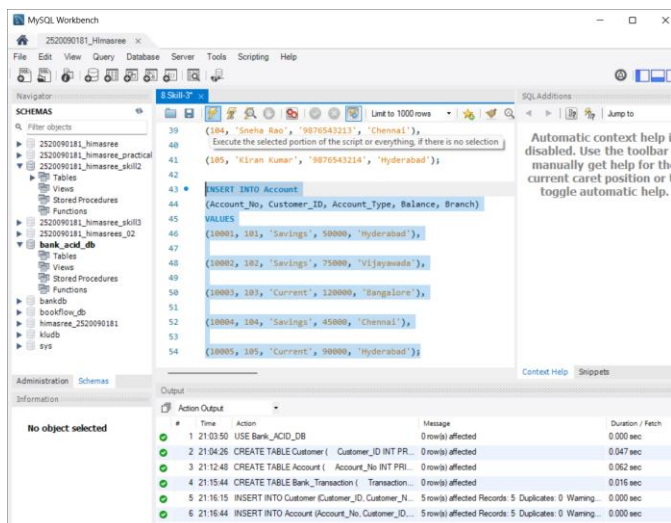
4. CREATE TRANSACTION TABLE



5. INSERT CUSTOMER DATA



6. INSERT ACCOUNT DATA



7. INSERT BANK TRANSACTION DATA

The screenshot shows the MySQL Workbench interface with a SQL script in the editor. The script contains several INSERT statements into the Bank_Transaction table. The Output pane at the bottom shows the execution results, indicating that the data was successfully inserted.

```
1 21:03:50 USE Bank_ACD_DB
2 21:04:26 CREATE TABLE Customer ( Customer_ID INT PRI
3 21:12:48 CREATE TABLE Account ( Account_No INT PRI
4 21:15:44 CREATE TABLE Bank_Transaction ( Transaction_
5 21:16:15 INSERT INTO Customer (Customer_ID, Customer_N
6 21:16:44 INSERT INTO Account (Account_No, Customer_ID,
7 21:17:29 INSERT INTO Bank_Transaction (Account_No, Tra
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

8. DISPLAY DATA

The screenshot shows the MySQL Workbench interface with a SQL script in the editor. The script contains several SELECT statements to display data from the Customer, Account, and Bank_Transaction tables. The Output pane at the bottom shows the execution results, indicating that the data was successfully displayed.

```
66 (10005, 'WITHDRAWAL', 10000);
67 SELECT * FROM Customer;
68
69 SELECT * FROM Account;
70
71 SELECT * FROM Bank_Transaction;
```

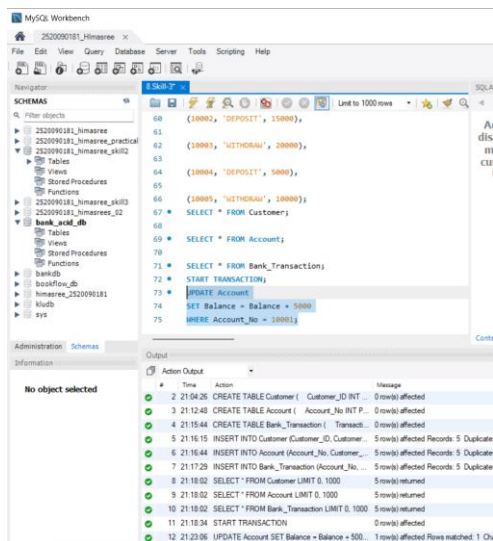
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

9. START A TRANSACTION

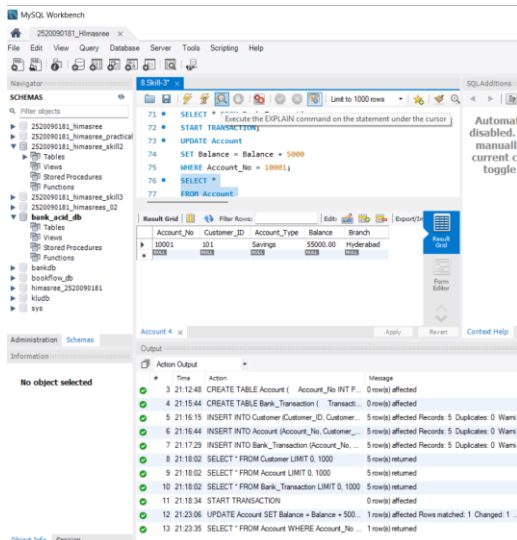
The screenshot shows the MySQL Workbench interface with a SQL script in the editor. The script contains several statements, including a START TRANSACTION statement. The Output pane at the bottom shows the execution results, indicating that the transaction was successfully started.

```
1 21:03:50 USE Bank_ACD_DB
2 21:04:26 CREATE TABLE Customer ( Customer_ID INT PRI
3 21:12:48 CREATE TABLE Account ( Account_No INT PRI
4 21:15:44 CREATE TABLE Bank_Transaction ( Transaction_
5 21:16:15 INSERT INTO Customer (Customer_ID, Customer_N
6 21:16:44 INSERT INTO Account (Account_No, Customer_ID,
7 21:17:29 INSERT INTO Bank_Transaction (Account_No, Tra
8 21:18:02 SELECT * FROM Customer LIMIT 0.1000
9 21:18:02 SELECT * FROM Account LIMIT 0.1000
10 21:18:02 SELECT * FROM Bank_Transaction LIMIT 0.1000
11 START TRANSACTION
```

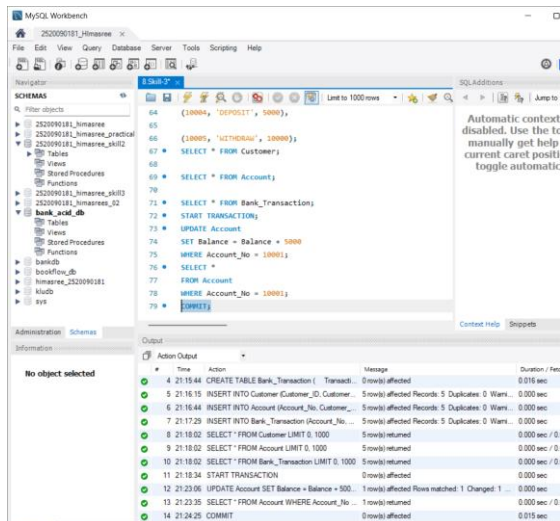
10. DEPOSIT MONEY



11. CHECK BALANCE



12. COMMIT TRANSACTION



13. CHECK BALANCE AFTER COMMIT

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
75 WHERE Account_No = 10001;  
76 SELECT *  
77 FROM Account  
78 WHERE Account_No = 10001;  
79 COMMIT;  
80 SELECT *  
81 FROM Account
```

The Results pane shows a table with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	5000.00	Hyderabad
10001	1001	1001	1001	1001

The Action Output pane shows the following messages:

- 5 21:16:15 INSERT INTO Customer (Customer_ID, Customer_Name, Address, Contact_No) VALUES (1001, 'Kumar', '123 Main St', 9876543210) 5 row(s) affected Records: 5 Duplicate
- 6 21:16:44 INSERT INTO Account (Account_No, Customer_No, Balance, Branch) VALUES (10001, 1001, 5000.00, 'Hyderabad') 5 row(s) affected Records: 5 Duplicate
- 7 21:17:29 INSERT INTO Bank_Transaction (Account_No, Customer_No, Transaction_No, Amount, Transaction_Type) VALUES (10001, 1001, 1001, 1000, 'Deposit') 5 row(s) affected Records: 5 Duplicate
- 8 21:18:02 SELECT * FROM Customer LIMIT 0, 1000 5 row(s) returned
- 9 21:18:02 SELECT * FROM Account LIMIT 0, 1000 5 row(s) returned
- 10 21:18:02 SELECT * FROM Bank_Transaction LIMIT 0, 1000 5 row(s) returned
- 11 21:18:34 START TRANSACTION 0 row(s) affected
- 12 21:23:06 UPDATE Account SET Balance = Balance + 500. 1 row(s) affected Rows matched: 1 Ch
- 13 21:23:35 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned
- 14 21:24:25 COMMIT 0 row(s) affected
- 15 21:25:25 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned

14. START TRANSACTION

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
68 SELECT * FROM Account;  
69  
70  
71 SELECT * FROM Bank_Transaction;  
72 START TRANSACTION;  
73 UPDATE Account  
74 SET Balance = Balance + 5000  
75 WHERE Account_No = 10001;  
76 SELECT *  
77 FROM Account  
78 WHERE Account_No = 10001;  
79 COMMIT;  
80 SELECT *  
81 FROM Account  
82 WHERE Account_No = 10001;  
83 START TRANSACTION
```

The Results pane shows a table with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	5000.00	Hyderabad
10001	1001	1001	1001	1001

The Action Output pane shows the following messages:

- 6 21:16:44 INSERT INTO Account (Account_No, Customer_No, Balance, Branch) VALUES (10001, 1001, 5000.00, 'Hyderabad') 5 row(s) affected Records: 5 Dupl
- 7 21:17:29 INSERT INTO Bank_Transaction (Account_No, Customer_No, Transaction_No, Amount, Transaction_Type) VALUES (10001, 1001, 1001, 1000, 'Deposit') 5 row(s) affected Records: 5 Dupl
- 8 21:18:02 SELECT * FROM Customer LIMIT 0, 1000 5 row(s) returned
- 9 21:18:02 SELECT * FROM Account LIMIT 0, 1000 5 row(s) returned
- 10 21:18:02 SELECT * FROM Bank_Transaction LIMIT 0, 1000 5 row(s) returned
- 11 21:18:34 START TRANSACTION 0 row(s) affected
- 12 21:23:06 UPDATE Account SET Balance = Balance + 500. 1 row(s) affected Rows matched: 1
- 13 21:23:35 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned
- 14 21:24:25 COMMIT 0 row(s) affected
- 15 21:25:25 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned
- 16 21:25:52 START TRANSACTION 0 row(s) affected

15. WITHDRAW MONEY

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
71 SELECT * FROM Bank_Transaction;  
72 START TRANSACTION;  
73 UPDATE Account  
74 SET Balance = Balance + 5000  
75 WHERE Account_No = 10001;  
76 SELECT *  
77 FROM Account  
78 WHERE Account_No = 10001;  
79 COMMIT;  
80 SELECT *  
81 FROM Account  
82 WHERE Account_No = 10001;  
83 START TRANSACTION  
84 UPDATE Account  
85 SET Balance = Balance - 10000  
86 WHERE Account_No = 10001;
```

The Results pane shows a table with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	5000.00	Hyderabad
10001	1001	1001	1001	1001

The Action Output pane shows the following messages:

- 7 21:17:29 INSERT INTO Bank_Transaction (Account_No, Customer_No, Transaction_No, Amount, Transaction_Type) VALUES (10001, 1001, 1001, 1000, 'Deposit') 5 row(s) affected Records: 5 Dupl
- 8 21:18:02 SELECT * FROM Customer LIMIT 0, 1000 5 row(s) returned
- 9 21:18:02 SELECT * FROM Account LIMIT 0, 1000 5 row(s) returned
- 10 21:18:02 SELECT * FROM Bank_Transaction LIMIT 0, 1000 5 row(s) returned
- 11 21:18:34 START TRANSACTION 0 row(s) affected
- 12 21:23:06 UPDATE Account SET Balance = Balance + 500. 1 row(s) affected Rows matched: 1
- 13 21:23:35 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned
- 14 21:24:25 COMMIT 0 row(s) affected
- 15 21:25:25 SELECT * FROM Account WHERE Account_No = 10001 1 row(s) returned
- 16 21:25:52 START TRANSACTION 0 row(s) affected
- 17 21:26:25 UPDATE Account SET Balance = Balance - 100. 1 row(s) affected Rows matched: 1

16. CHECK BALANCE

The screenshot shows a MySQL Workbench window with a SQL script in the editor. The script includes a WHERE clause, a START TRANSACTION statement, an UPDATE statement, a SET statement, a WHERE clause, and a SELECT statement. The output pane displays a table with columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The table contains two rows of data.

```
82 WHERE Account_No = 10001;
83 START TRANSACTION;
84 UPDATE Account
85 SET Balance = Balance - 10000
86 WHERE Account_No = 10001;
87 SELECT *
88 FROM Account;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	45000.00	Hyderabad
10008	009	0008	0008	

17. ROLLBACK

The screenshot shows a MySQL Workbench window with a SQL script in the editor. The script includes a WHERE clause, a SELECT statement, a FROM clause, a WHERE clause, a COMMIT statement, a FROM clause, a WHERE clause, a START TRANSACTION statement, an UPDATE statement, a SET statement, a WHERE clause, a SELECT statement, a FROM clause, a WHERE clause, and a ROLLBACK statement. The output pane displays a table with columns: #, Time, Action, and Message. The table contains 20 rows of data, including the ROLLBACK statement.

```
75 WHERE Account_No = 10001;
76 SELECT *
77 FROM Account
78 WHERE Account_No = 10001;
79 COMMIT;
80 SELECT *
81 FROM Account
82 WHERE Account_No = 10001;
83 START TRANSACTION;
84 UPDATE Account
85 SET Balance = Balance - 10000
86 WHERE Account_No = 10001;
87 SELECT *
88 FROM Account
89 WHERE Account_No = 10001;
90 ROLLBACK;
```

#	Time	Action	Message
8	21:18:02	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
9	21:18:02	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
10	21:18:02	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
11	21:18:04	START TRANSACTION	0 row(s) affected
12	21:23:06	UPDATE Account SET Balance = Balance - 500.	1 row(s) affected Rows matched: 1 Changed: 1
13	21:23:35	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned
14	21:24:25	COMMIT	0 row(s) affected
15	21:25:25	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned
16	21:25:52	START TRANSACTION	0 row(s) affected
17	21:26:29	UPDATE Account SET Balance = Balance - 100.	1 row(s) affected Rows matched: 1 Changed: 1
18	21:27:45	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned
19	21:28:08	WHERE Account_No = 10001	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE Account_No = 10001' at line 1
20	21:28:12	ROLLBACK	0 row(s) affected

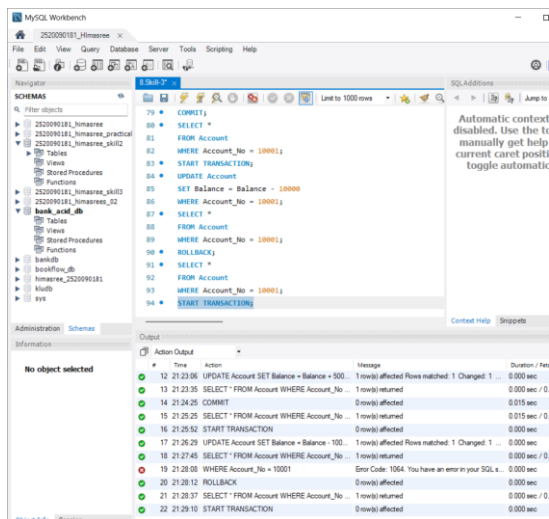
18. CHECK BALANCE AFTER ROLLBACK

The screenshot shows a MySQL Workbench window with a SQL script in the editor. The script includes a WHERE clause, a SELECT statement, a FROM clause, a WHERE clause, a ROLLBACK statement, a SELECT statement, and a FROM clause. The output pane displays a table with columns: #, Time, Action, Message, and Duration / Row. The table contains 21 rows of data, including the ROLLBACK statement.

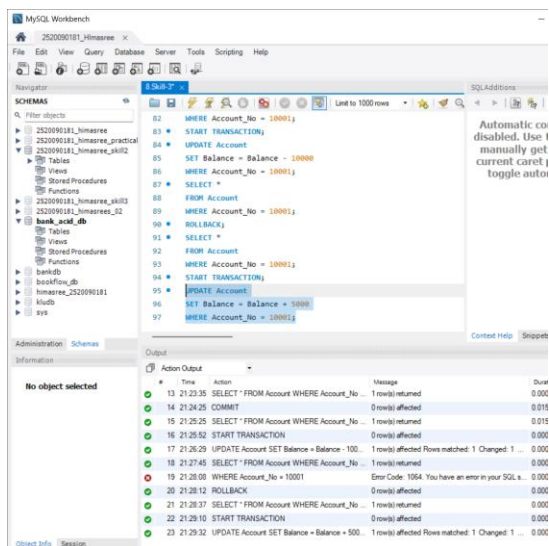
```
86 WHERE Account_No = 10001;
87 SELECT *
88 FROM Account
89 WHERE Account_No = 10001;
90 ROLLBACK;
91 SELECT *
92 FROM Account;
```

#	Time	Action	Message	Duration / Row
11	21:18:04	START TRANSACTION	0 row(s) affected	0.000 sec
12	21:23:06	UPDATE Account SET Balance = Balance - 500.	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
13	21:23:35	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned	0.000 sec / 0.000 sec
14	21:24:25	COMMIT	0 row(s) affected	0.015 sec
15	21:25:25	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned	0.015 sec / 0.000 sec
16	21:25:52	START TRANSACTION	0 row(s) affected	0.000 sec
17	21:26:29	UPDATE Account SET Balance = Balance - 100.	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
18	21:27:45	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned	0.000 sec / 0.000 sec
19	21:28:08	WHERE Account_No = 10001	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE Account_No = 10001' at line 1	0.000 sec
20	21:28:12	ROLLBACK	0 row(s) affected	0.000 sec
21	21:28:37	SELECT * FROM Account WHERE Account_No = 10001	1 row(s) returned	0.000 sec / 0.000 sec

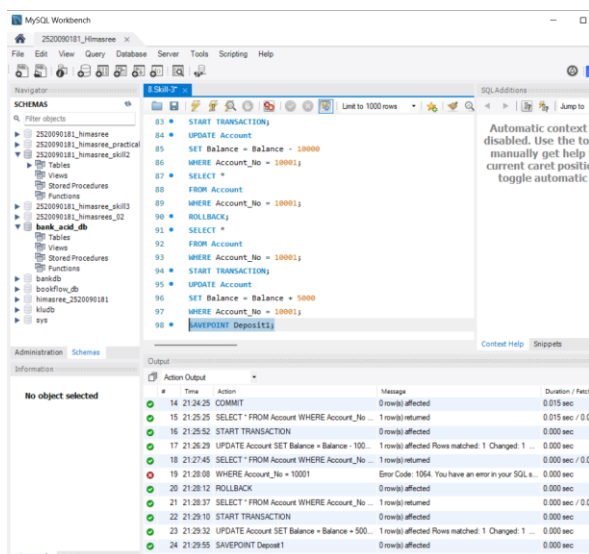
19. START TRANSACTION



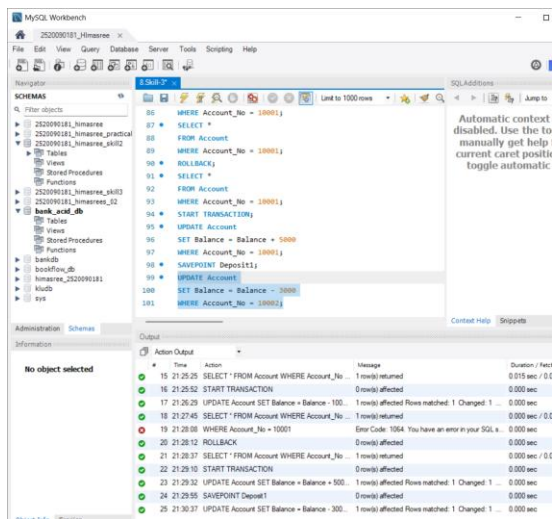
20. FIRST UPDATE



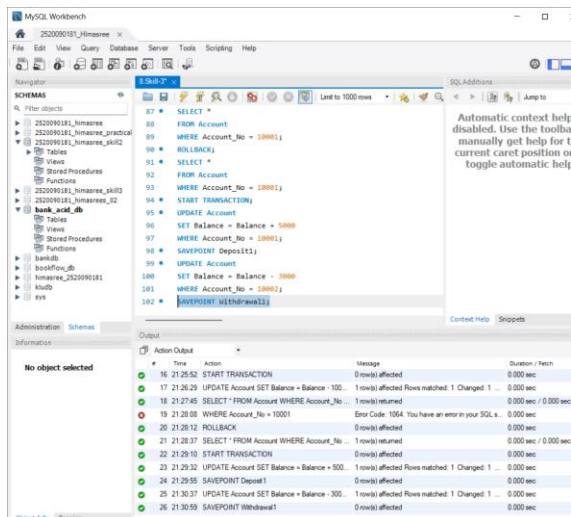
21. CREATE SAVEPOINT



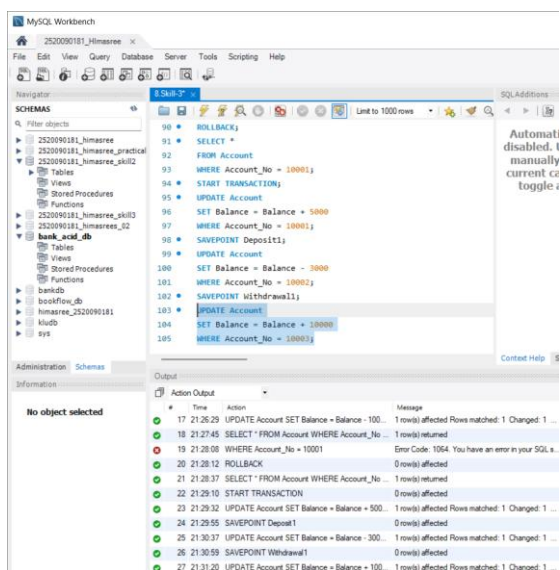
22. SECOND UPDATE



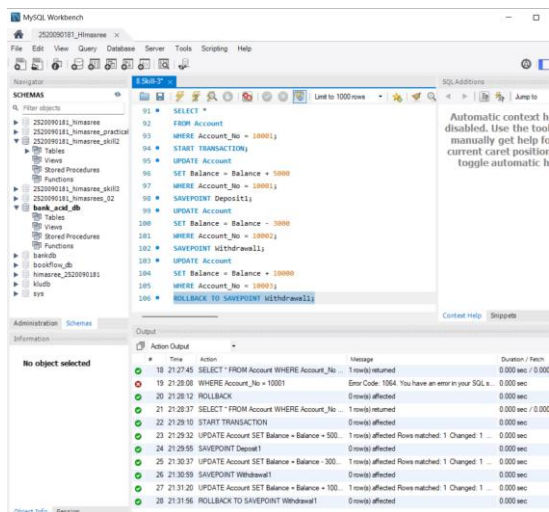
23. CREATE SECOND SAVEPOINT



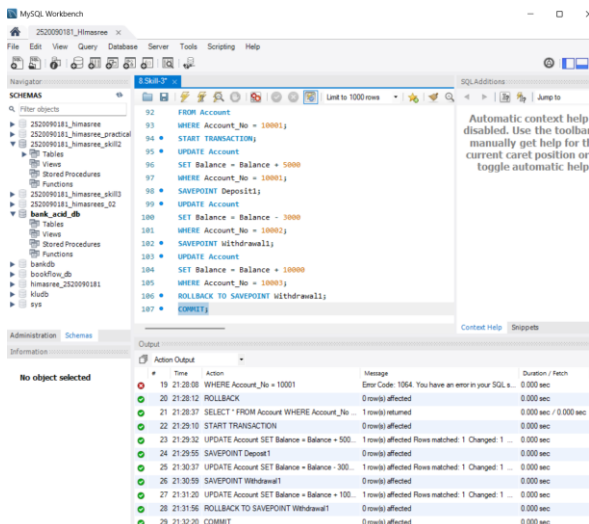
24. THIRD UPDATE



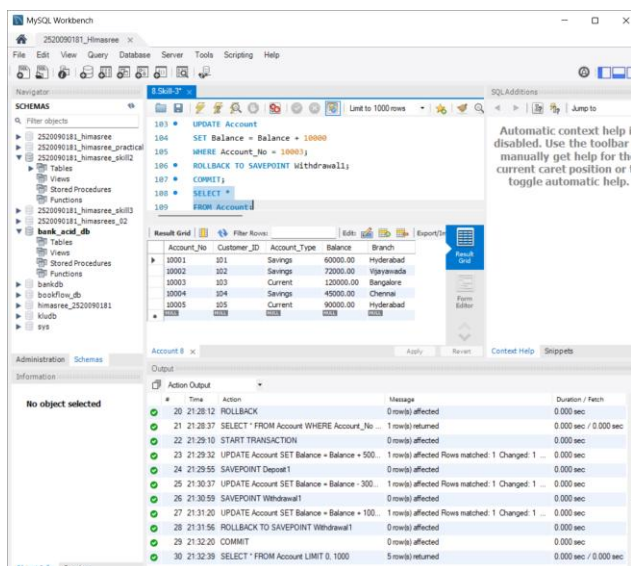
25. ROLLBACK TO SAVEPOINT



26. COMMIT



27. CHECK ACCOUNTS



28. TRANSFER MONEY

MySQL Workbench interface showing a SQL script for a money transfer transaction. The script is as follows:

```
96 SET Balance = Balance + 5000
97 WHERE Account_No = 10001;
98 SAVEPOINT Deposit1;
99 UPDATE Account
100 SET Balance = Balance - 3000
101 WHERE Account_No = 10001;
102 SAVEPOINT Withdrawal1;
103 UPDATE Account
104 SET Balance = Balance + 10000
105 WHERE Account_No = 10001;
106 ROLLBACK TO SAVEPOINT Withdrawal1;
107 COMMIT;
108 SELECT *
109 FROM Account;
110 START TRANSACTION;
111
```

Automatic context help disabled. Use the toolbar manually get help for the current caret position or toggle automatic help.

#	Time	Action	Message	Duration / Fetch
21	21:29:37	SELECT * FROM Account WHERE Account_No = 10001	5 row(s) returned	0.000 sec / 0.000 sec
22	21:29:10	START TRANSACTION	0 row(s) affected	0.000 sec
23	21:29:32	UPDATE Account SET Balance = Balance + 5000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
24	21:29:55	SAVEPOINT Deposit1	0 row(s) affected	0.000 sec
25	21:30:37	UPDATE Account SET Balance = Balance - 3000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
26	21:30:59	SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
27	21:31:20	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
28	21:31:56	ROLLBACK TO SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec

29. DEDUCT FROM SENDER

MySQL Workbench interface showing a SQL script for deducting money from the sender. The script is as follows:

```
98 SAVEPOINT Deposit1;
99 UPDATE Account
100 SET Balance = Balance - 3000
101 WHERE Account_No = 10001;
102 SAVEPOINT Withdrawal1;
103 UPDATE Account
104 SET Balance = Balance + 10000
105 WHERE Account_No = 10001;
106 ROLLBACK TO SAVEPOINT Withdrawal1;
107 COMMIT;
108 SELECT *
109 FROM Account;
110 START TRANSACTION;
111 UPDATE Account
112 SET Balance = Balance - 10000
113 WHERE Account_No = 10001;
114
```

Automatic context help disabled. Use the toolbar manually get help for the current caret position or toggle automatic help.

#	Time	Action	Message	Duration / Fetch
22	21:29:10	START TRANSACTION	0 row(s) affected	0.000 sec
23	21:29:32	UPDATE Account SET Balance = Balance + 5000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
24	21:29:55	SAVEPOINT Deposit1	0 row(s) affected	0.000 sec
25	21:30:37	UPDATE Account SET Balance = Balance - 3000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
26	21:30:59	SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
27	21:31:20	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
28	21:31:56	ROLLBACK TO SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec

30. ADD TO RECEIVER

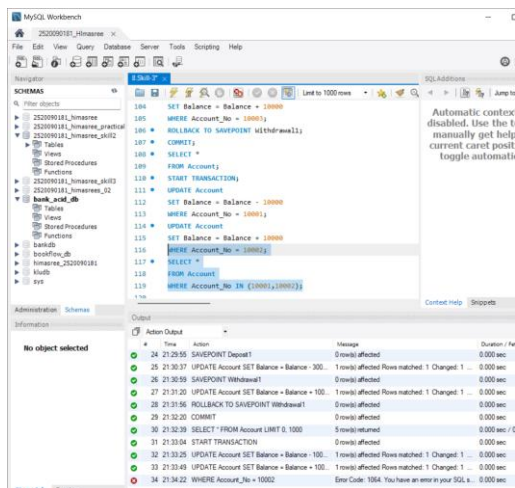
MySQL Workbench interface showing a SQL script for adding money to the receiver. The script is as follows:

```
102 SAVEPOINT Withdrawal1;
103 UPDATE Account
104 SET Balance = Balance - 10000
105 WHERE Account_No = 10001;
106 ROLLBACK TO SAVEPOINT Withdrawal1;
107 COMMIT;
108 SELECT *
109 FROM Account;
110 START TRANSACTION;
111 UPDATE Account
112 SET Balance = Balance + 10000
113 WHERE Account_No = 10001;
114 UPDATE Account
115 SET Balance = Balance + 10000
116 WHERE Account_No = 10001;
117
```

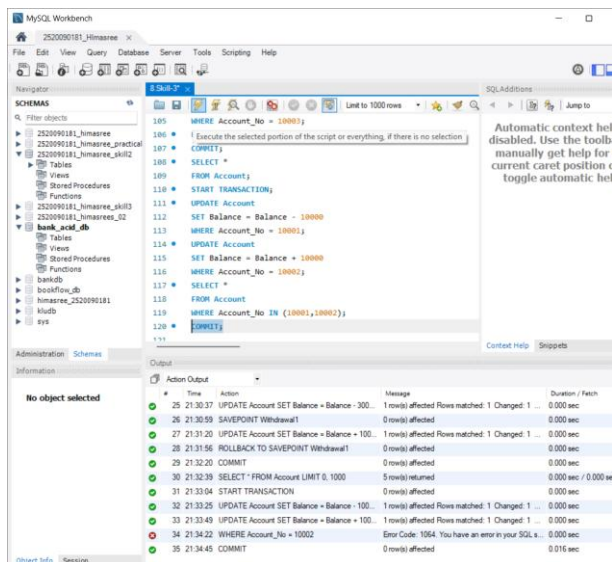
Automatic context help disabled. Use the toolbar manually get help for the current caret position or toggle automatic help.

#	Time	Action	Message	Duration / Fetch
23	21:29:32	UPDATE Account SET Balance = Balance + 5000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
24	21:29:55	SAVEPOINT Deposit1	0 row(s) affected	0.000 sec
25	21:30:37	UPDATE Account SET Balance = Balance - 3000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
26	21:30:59	SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
27	21:31:20	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
28	21:31:56	ROLLBACK TO SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
33	21:33:43	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec

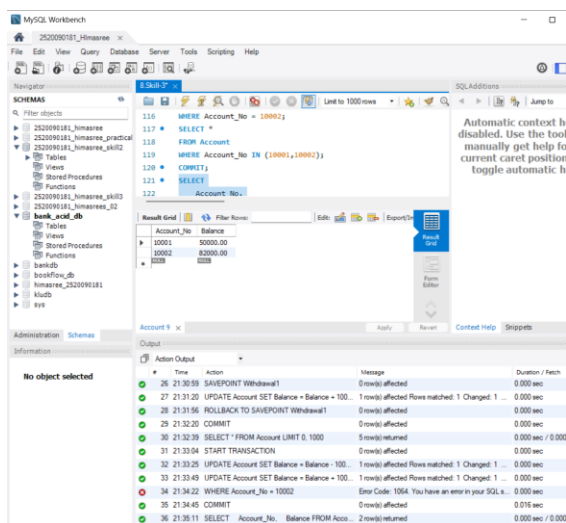
31. CHECK BOTH ACCOUNTS



32. COMMIT TRANSFER



33. FINAL BALANCES



34. START TRANSACTION

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following script:

```
111 UPDATE Account
112 SET Balance = Balance - 10000
113 WHERE Account_No = 10002;
114 UPDATE Account
115 SET Balance = Balance + 10000
116 WHERE Account_No = 10002;
117 SELECT *
118 FROM Account
119 WHERE Account_No IN (10001,10002);
120 COMMIT;
121 SELECT
122     Account_No,
123     Balance
124 FROM Account
125 WHERE Account_No IN (10001,10002);
126 START TRANSACTION;
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fech
27	21:31:20	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
28	21:31:56	ROLLBACK TO SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
33	21:33:49	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
34	21:34:22	WHERE Account_No = 10002	Error Code: 1064: You have an error in your SQL s...	0.000 sec
35	21:34:45	COMMIT	0 row(s) affected	0.016 sec
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000 sec
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 20000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec

35. DEDUCT MONEY

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following script:

```
114 UPDATE Account
115 SET Balance = Balance + 10000
116 WHERE Account_No = 10002;
117 SELECT *
118 FROM Account
119 WHERE Account_No IN (10001,10002);
120 COMMIT;
121 SELECT
122     Account_No,
123     Balance
124 FROM Account
125 WHERE Account_No IN (10001,10002);
126 START TRANSACTION;
127 UPDATE Account
128 SET Balance = Balance - 20000
129 WHERE Account_No = 10001;
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fech
28	21:31:56	ROLLBACK TO SAVEPOINT Withdrawal1	0 row(s) affected	0.000 sec
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
33	21:33:49	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
34	21:34:22	WHERE Account_No = 10002	Error Code: 1064: You have an error in your SQL s...	0.000 sec
35	21:34:45	COMMIT	0 row(s) affected	0.016 sec
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000 sec
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 20000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec

36. ADD MONEY

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following script:

```
117 SELECT *
118 FROM Account
119 WHERE Account_No IN (10001,10002);
120 COMMIT;
121 SELECT
122     Account_No,
123     Balance
124 FROM Account
125 WHERE Account_No IN (10001,10002);
126 START TRANSACTION;
127 UPDATE Account
128 SET Balance = Balance - 20000
129 WHERE Account_No = 10001;
130 UPDATE Account
131 SET Balance = Balance + 20000
132 WHERE Account_No = 10002;
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fech
29	21:32:20	COMMIT	0 row(s) affected	0.000 sec
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
33	21:33:49	UPDATE Account SET Balance = Balance + 10000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
34	21:34:22	WHERE Account_No = 10002	Error Code: 1064: You have an error in your SQL s...	0.000 sec
35	21:34:45	COMMIT	0 row(s) affected	0.016 sec
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000 sec
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 20000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
39	21:36:16	UPDATE Account SET Balance = Balance + 20000	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec

37. CANCEL TRANSACTION

MySQL Workbench interface showing a transaction being rolled back. The SQL editor contains the following code:

```
118 FROM Account
119 WHERE Account_No IN (10001,10002);
120 * COMMIT;
121 * SELECT
122     Account_No,
123     Balance
124 FROM Account
125 WHERE Account_No IN (10001,10002);
126 * START TRANSACTION;
127 * UPDATE Account
128 SET Balance = Balance - 20000
129 WHERE Account_No = 10001;
130 * UPDATE Account
131 SET Balance = Balance + 20000
132 WHERE Account_No = 10002;
133 * ROLLBACK;
```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fech
30	21:32:39	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
33	21:33:45	UPDATE Account SET Balance = Balance + 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
34	21:34:02	WHERE Account_No = 10002	Error Code: 1064: You have an error in your SQL s...	0.000 sec
35	21:34:45	COMMIT	0 row(s) affected	0.016 sec
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
39	21:36:16	UPDATE Account SET Balance = Balance + 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
40	21:36:55	ROLLBACK	0 row(s) affected	0.000 sec

38. CHECK BALANCES

MySQL Workbench interface showing account balances. The SQL editor contains the following code:

```
129 WHERE Account_No = 10001;
130 * UPDATE Account
131 SET Balance = Balance + 20000
132 WHERE Account_No = 10002;
133 * ROLLBACK;
134 * SELECT *
135 FROM Account;
```

The Result Grid shows the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	82000.00	Vijayawada

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fech
31	21:33:04	START TRANSACTION	0 row(s) affected	0.000 sec
32	21:33:25	UPDATE Account SET Balance = Balance - 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
33	21:33:45	UPDATE Account SET Balance = Balance + 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
34	21:34:22	WHERE Account_No = 10002	Error Code: 1064: You have an error in your SQL s...	0.000 sec
35	21:34:45	COMMIT	0 row(s) affected	0.016 sec
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
39	21:36:16	UPDATE Account SET Balance = Balance + 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
40	21:36:55	ROLLBACK	0 row(s) affected	0.000 sec
41	21:37:14	SELECT * FROM Account WHERE Account_No...	2 row(s) returned	0.000 sec / 0.000

39. ATOMICITY

MySQL Workbench interface showing account balances. The SQL editor contains the following code:

```
130 * UPDATE Account
131 SET Balance = Balance + 20000
132 WHERE Account_No = 10002;
133 * ROLLBACK;
134 * SELECT *
135 FROM Account;
```

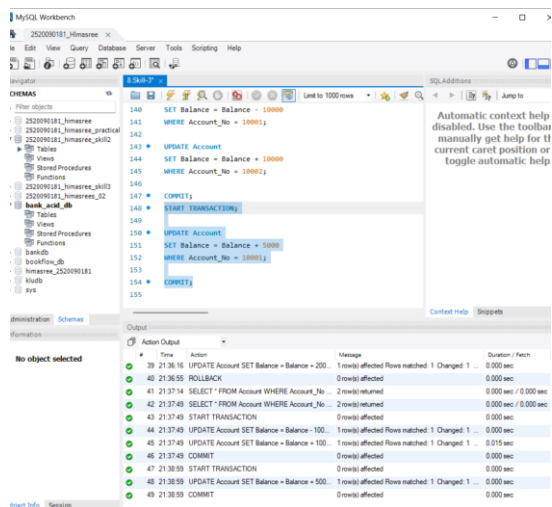
The Result Grid shows the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	82000.00	Vijayawada

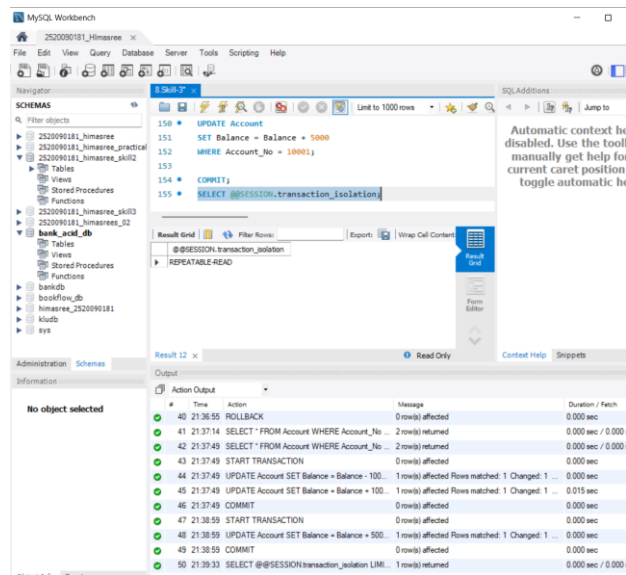
The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fech
36	21:35:11	SELECT Account_No, Balance FROM Acco...	2 row(s) returned	0.000 sec / 0.000
37	21:35:33	START TRANSACTION	0 row(s) affected	0.000 sec
38	21:35:55	UPDATE Account SET Balance = Balance - 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
39	21:36:16	UPDATE Account SET Balance = Balance + 200;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
40	21:36:55	ROLLBACK	0 row(s) affected	0.000 sec
41	21:37:14	SELECT * FROM Account WHERE Account_No...	2 row(s) returned	0.000 sec / 0.000
42	21:37:49	SELECT * FROM Account WHERE Account_No...	2 row(s) returned	0.000 sec / 0.000
43	21:37:49	START TRANSACTION	0 row(s) affected	0.000 sec
44	21:37:49	UPDATE Account SET Balance = Balance - 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
45	21:37:49	UPDATE Account SET Balance = Balance - 100;	1 row(s) affected Rows matched: 1 Changed: 1	0.016 sec
46	21:37:49	COMMIT	0 row(s) affected	0.000 sec

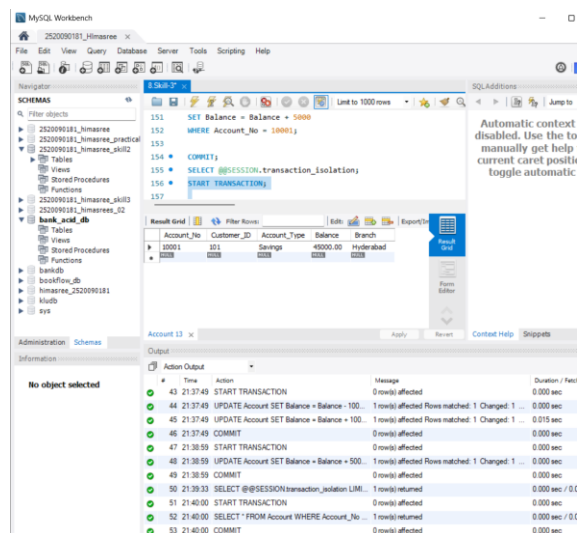
42. DURABILITY



43. CHECK CURRENT ISOLATION LEVEL



44. READ UNCOMMITTED



45. READ COMMITTED

MySQL Workbench interface showing the Read Committed isolation level. The SQL editor contains the following queries:

```
158 FROM Account
161 WHERE Account_No = 10001;
162
164 * COMMIT;
165 * START TRANSACTION;
```

The Result Grid shows one row:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	45000.00	Hyderabad

The Action Output pane shows the execution log:

Time	Action	Message	Duration
46 21:37:43	COMMIT	0 rows affected	0.000 sec
47 21:38:59	START TRANSACTION	0 rows affected	0.000 sec
48 21:38:59	UPDATE Account SET Balance = Balance + 500.	1 row(s) affected Rows matched: 1 Changed: 1	0.000 sec
49 21:38:59	COMMIT	0 rows affected	0.000 sec
50 21:39:33	SELECT @@SESSION.transaction_isolation (RM)	1 row(s) returned	0.000 sec
51 21:40:00	START TRANSACTION	0 rows affected	0.000 sec
52 21:40:00	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec
53 21:40:00	COMMIT	0 rows affected	0.000 sec
54 21:40:40	START TRANSACTION	0 rows affected	0.015 sec
55 21:40:40	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec
56 21:40:40	COMMIT	0 rows affected	0.000 sec

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

46. REPEATABLE READ

MySQL Workbench interface showing the Repeatable Read isolation level. The SQL editor contains the following queries:

```
169 FROM Account
170 WHERE Account_No = 10001;
171
173 * COMMIT;
174 * START TRANSACTION;
```

The Result Grid shows one row:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	45000.00	Hyderabad

The Action Output pane shows the execution log:

Time	Action	Message	Duration
49 21:38:59	COMMIT	0 rows affected	0.000 sec
50 21:39:33	SELECT @@SESSION.transaction_isolation (RM)	1 row(s) returned	0.000 sec
51 21:40:00	START TRANSACTION	0 rows affected	0.000 sec
52 21:40:00	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec
53 21:40:00	COMMIT	0 rows affected	0.015 sec
54 21:40:40	START TRANSACTION	1 row(s) returned	0.000 sec
55 21:40:40	COMMIT	0 rows affected	0.000 sec
56 21:40:40	COMMIT	0 rows affected	0.000 sec
57 21:41:04	START TRANSACTION	0 rows affected	0.000 sec
58 21:41:04	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec
59 21:41:05	COMMIT	0 rows affected	0.000 sec

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

47. SERIALIZABLE

MySQL Workbench interface showing the Serializable isolation level. The SQL editor contains the following queries:

```
176 FROM Account
179 WHERE Account_No = 10001;
180
182 * COMMIT;
183 * START TRANSACTION;
```

The Result Grid shows one row:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	45000.00	Hyderabad

The Action Output pane shows the execution log:

Time	Action	Message	Duration
52 21:40:00	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.0
53 21:40:00	COMMIT	0 rows affected	0.0
54 21:40:40	START TRANSACTION	0 rows affected	0.0
55 21:40:40	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.0
56 21:40:40	COMMIT	0 rows affected	0.0
57 21:41:04	START TRANSACTION	0 rows affected	0.0
58 21:41:04	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.0
59 21:41:05	COMMIT	0 rows affected	0.0
60 21:41:43	START TRANSACTION	0 rows affected	0.0
61 21:41:43	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.0
62 21:41:43	COMMIT	0 rows affected	0.0

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

48. DEPOSIT TRANSACTION

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

```

184
185
186 * SELECT *
187   FROM Account
188   WHERE Account_No = 10001;
189
190
191 * COMMIT;
192
193 * START TRANSACTION;
194
195 * UPDATE Account
196   SET Balance = Balance + 20000
197   WHERE Account_No = 10001;
198
199

```

#	Time	Action	Message	Duration / Rows
54	21:40:40	START TRANSACTION	0 rows affected	0.015 s
55	21:40:40	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 s
56	21:40:40	COMMIT	0 row(s) affected	0.000 s
57	21:41:04	START TRANSACTION	0 row(s) affected	0.000 s
58	21:41:04	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 s
59	21:41:05	COMMIT	0 row(s) affected	0.000 s
60	21:41:43	START TRANSACTION	0 row(s) affected	0.000 s
61	21:41:43	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 s
62	21:41:43	COMMIT	0 row(s) affected	0.000 s
63	21:42:09	START TRANSACTION	0 row(s) affected	0.000 s
64	21:42:09	UPDATE Account SET Balance = Balance + 200...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 s

49. WITHDRAWAL TRANSACTION

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

```

192 * START TRANSACTION;
193
194 * UPDATE Account
195   SET Balance = Balance - 20000
196   WHERE Account_No = 10001;
197
198 * COMMIT;
199

```

#	Time	Action	Message	Duration / Rows
56	21:40:40	COMMIT	0 row(s) affected	0.000 sec
57	21:41:04	START TRANSACTION	0 row(s) affected	0.000 sec
58	21:41:04	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec / 1
59	21:41:05	COMMIT	0 row(s) affected	0.000 sec
60	21:41:43	START TRANSACTION	0 row(s) affected	0.000 sec
61	21:41:43	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec / 1
62	21:41:43	COMMIT	0 row(s) affected	0.000 sec
63	21:42:09	START TRANSACTION	0 row(s) affected	0.000 sec
64	21:42:09	UPDATE Account SET Balance = Balance + 200...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
65	21:42:35	START TRANSACTION	0 row(s) affected	0.000 sec
66	21:42:35	SELECT Balance FROM Account WHERE Acco...	1 row(s) returned	0.000 sec / 1

50. CANCEL WITHDRAWAL

Automatic context disabled. Use the manually get the current caret position to toggle automatic.

```

194 * UPDATE Account
195   SET Balance = Balance + 5000
196   WHERE Account_No = 10001;
197
198 * START TRANSACTION;
199
200 * SELECT Balance
201   FROM Account
202   WHERE Account_No = 10001;
203
204 * UPDATE Account
205   SET Balance = Balance + 5000
206   WHERE Account_No = 10001;
207
208 * COMMIT;
209

```

#	Time	Action	Message	Duration / Rows
59	21:41:05	COMMIT	0 row(s) affected	0.000 sec
60	21:41:43	START TRANSACTION	0 row(s) affected	0.000 sec
61	21:41:43	SELECT * FROM Account WHERE Account_No ...	1 row(s) returned	0.000 sec / 0
62	21:41:43	COMMIT	0 row(s) affected	0.000 sec
63	21:42:09	START TRANSACTION	0 row(s) affected	0.000 sec
64	21:42:09	UPDATE Account SET Balance = Balance + 200...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
65	21:42:35	START TRANSACTION	0 row(s) affected	0.000 sec
66	21:42:35	SELECT Balance FROM Account WHERE Acco...	1 row(s) returned	0.000 sec / 0
67	21:43:14	START TRANSACTION	0 row(s) affected	0.000 sec
68	21:43:14	UPDATE Account SET Balance = Balance + 500...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
69	21:43:14	COMMIT	0 row(s) affected	0.000 sec

51. TRANSFER WITH SAVEPOINT

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with databases like '2520090181_himasree' and 'bank_acct_db'. The main editor contains a SQL script for a bank transfer. The script includes a transaction with a savepoint, an update, and a rollback to the savepoint.

```
207
208 • COMMIT;
209 • START TRANSACTION;
210
211 • UPDATE Account
212   SET Balance = Balance - 10000
213   WHERE Account_No = 100001;
214
215 • SAVEPOINT AfterDebit;
216
217 • UPDATE Account
218   SET Balance = Balance + 10000
219   WHERE Account_No = 100002;
220
221 • COMMIT;
222
```

Below the script, the 'Output' tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
64	21:42:09	UPDATE Account SET Balance = Balance + 200...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
65	21:42:35	START TRANSACTION	0 row(s) affected	0.000 sec
66	21:42:35	SELECT Balance FROM Account WHERE Acco...	1 row(s) returned	0.000 sec / 0.000
67	21:43:14	START TRANSACTION	0 row(s) affected	0.000 sec
68	21:43:14	UPDATE Account SET Balance = Balance + 500...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
69	21:43:14	COMMIT	0 row(s) affected	0.000 sec
70	21:44:00	START TRANSACTION	0 row(s) affected	0.000 sec
71	21:44:00	UPDATE Account SET Balance = Balance - 100...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
72	21:44:00	SAVEPOINT AfterDebit	0 row(s) affected	0.000 sec
73	21:44:00	UPDATE Account SET Balance = Balance + 100...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
74	21:44:00	COMMIT	0 row(s) affected	0.000 sec

On the right side of the editor, a note states: 'Automatic context h disabled. Use the tool manually get help fo current caret position toggle automatic h'.